Information Capacity in Neural Networks

An Empirical Study of κ Scaling

JoonHa Park
Independent Researcher

May 12, 2026

Abstract

We measure how information capacity varies with architecture, width, depth, and activation function in neural networks trained on CIFAR-10. Using an InfoNCE-based estimator across eight architectures (MLP, CNN, GRU, Pre-LN Transformer, Post-LN Transformer, Parallel-branch, Serial without residuals, restricted Boltzmann machine) under a structured Phase 0 measurement design over width, depth, activation, and seed, supplemented by targeted noise, temperature, dynamics, and phase-boundary sweeps, we record 2,241 measurement records and report four empirical observations plus one phenomenological scaling pattern.

We begin with a clarifying negative result: the activation function (ReLU, GeLU, tanh) changes κ_{input} by less than 1.5% in 7 of 8 architectures. The single exception is Post-LN Transformer with tanh (-10.6%), consistent with known training difficulties. Within this measurement scope, this suggests activation choice is not the dominant driver of the observed κ variation and motivates examination of structural variables. Four observations and one phenomenological pattern follow, ordered from strongest to most cautious.

(1) Phase-transition-like collapse (robust within scope). Architectures without residual connections exhibit width-dependent, phase-transition-like catastrophic collapse at critical depths d_c (Serial: w=256 $\rightarrow$ d_c=14; w=512 $\rightarrow$ d_c=8). Four independent metrics (κ, accuracy, train loss, sanity) agree on d_c, arguing that the collapse reflects training failure rather than only an estimator artifact.

(2) IB ratio (robust, d_z-canceling). The information bottleneck ratio $\eta_t = \kappa_{\text{task}}/\kappa_{\text{input}}$ — in which the d_z normalization cancels exactly between numerator and denominator (though shared InfoNCE estimator bias may still affect both terms) — correlates with test accuracy at $r = 0.685$ ($n = 633$, Figure 5), with $r > 0.65$ in 7 of 8 architectures.

(3) Training dynamics (robust). Within-network training trajectories show η_t rising during learning with mean within-config $r(\eta_t, \text{acc}) = +0.95$ across 15 configurations, providing a dynamic viewpoint consistent with the static IB ratio signal.

(4) Forward model and inverse design (proof-of-concept). A two-variable forward model $\kappa(w, d)$ extrapolates to held-out (w, d) cells with median 1.75% error in 5-fold cross-validation, serving as a proof-of-concept for inverse architecture design within the measured grid.

(5) Width scaling (phenomenological). Across the eight architectures, the per-record information capacity satisfies $\kappa \propto 1/w$ with $R^2 > 0.9999$ in width-averaged fits. We treat this as a *phenomenological observation in the present measurement regime*, not a universal scaling law: at batch size $B = 512$ the InfoNCE estimator approaches its finite-batch ceiling, so this $1/w$ pat-

tern reflects both architectural information content and per-dimension normalization ($d_z = w$). Disambiguating these requires larger-batch measurements (Phase 1).

We do not claim a finalized standard or a universal scaling law: all reported phenomena are within-scope observations under a fixed InfoNCE/noise/protocol regime. These observations motivate the companion ICON framework specification released alongside this paper. The present paper stands on its own as an empirical study; the four observations are reported as independent results. All data, fits, and analysis scripts are released with sha256-verified provenance; see Section 2.7 for the repository URL and verification hashes.

Contents

Introduction

Motivation: From Biological Intuition to Measurement

Biological neurons carry information through electrical signals, and those signals are bounded. Membrane physics, ion-channel kinetics, refractory periods, and metabolic budget all constrain how much information a single neuron can transmit per unit time. The neuroscience literature has characterized these limits quantitatively for decades.

Artificial neural networks are not biological systems. But they share an elementary structural feature: signals flow through layers of units, get transformed, and propagate forward. If biological neurons have measurable information-carrying limits, it is natural to ask whether the analogous quantity exists for artificial networks. *Can we directly measure how much information actually flows through an artificial neural network, and does it depend on architectural choices in measurable ways?* That intuition is the starting point of this paper. A philosophical companion essay (Park, 2026) develops the structural argument that *information flow*, as a vocabulary that abstracts away from substrate-specific implementation details, makes cross-substrate measurement approachable in principle; the present paper carries out the empirical work for the artificial-network substrate.

Information-theoretic analyses of neural networks have a long history. The information bottleneck principle (Tishby & Zaslavsky, 2015) provides a theoretical framework, and mutual information estimation methods (Belghazi et al., 2018; Oord et al., 2018; Poole et al., 2019) have made empirical measurement increasingly tractable. The quantity we study — mutual information per latent dimension, $\kappa = I(X; \tilde{Z})/d_z$ — is operationally a normalized estimate of an InfoNCE-bounded *channel capacity*: the network maps inputs X through the noisy channel $X \to \tilde{Z}$ (via the noise injection

of Section 2.1), and κ measures the per-dimension rate of that channel under our estimator regime. In parallel, neural scaling laws (Hestness et al., 2017; Kaplan et al., 2020; Hoffmann et al., 2022) have produced empirical regularities relating model size to loss. Yet direct measurement of how *information capacity* — mutual information per latent dimension — varies systematically across architectures, widths, and depths remains relatively unexplored. Most existing work either (i) studies a single architecture in isolation, or (ii) measures aggregate quantities (loss, accuracy) rather than information-theoretic ones.

Approaching the question fresh, we wanted to verify each step empirically rather than assume which variables would matter. The natural starting point was the architectural choice that every practitioner makes — the activation function. Conventional wisdom emphasizes activation choice as a first-order design decision; recent mechanistic work has documented systematic differences between ReLU, GeLU, and tanh in optimization dynamics. We measured. The activation function changes κ_{input} by less than 1.5% in 7 of 8 architectures studied here, at the canonical configuration (w=256, d=4; Section 4). This is a clarifying negative result: it suggests the most-studied architectural knob is not the dominant driver of κ variation at our measurement scope, and it directs attention to structural variables — width, depth, and the presence or absence of residual connections. Examining those variables yields the four observations that follow (Sections 4-9).

Our contribution is *methodological and empirical*, not theoretical: we measure, we observe, we report. We see the present empirical baseline as a starting point; the broader measurement framework that follow-up work develops — the companion ICON framework specification, released alongside this paper — is built on these observations.

Measurement Scope

We measure information capacity κ across eight neural architectures (MLP, CNN, GRU, Pre-LN Transformer, Post-LN Transformer, Parallel-branch, Serial without residuals, restricted Boltzmann machine) on CIFAR-10, under a structured Phase 0 measurement design over width w $\in$ {16, 32, 64, 128, 256, 512}, depth d $\in$ {1, 2, 4, 8, 16, 32}, three activation functions, and three random seeds, supplemented by targeted noise, temperature, dynamics, and phase-boundary sweeps rather than a complete Cartesian grid over all variables. The total is 2,241 measurement records (1,405 in the main results file + 836 in the phase-and-dynamics file), of which 633 pass our InfoNCE sanity criterion.

The methodology emphasizes four practical disciplines: (i) pre-specified filter conditions, aggregation rules, and statistical tests; (ii) reproducible provenance with sha256-hashed records and documented aggregations; (iii) explicit scope boundaries — single dataset, fixed batch size, $\leq$ 56M parameters — distinguished from larger-scope claims left for future work; (iv) honest interpretation of each observed regularity as one of *trivial, artifact, phenomenological,* or *robust* based on whether it survives controls.

Background note on ICON. The measurement methodology used here is the empirical basis for a more general framework, developed separately in follow-up work. The ICON specification defines five measurement components, a validity gate, and a settings discipline that generalize beyond the eight architectures studied here. The present paper does not require that framework: the four

observations reported below are independent empirical results. Readers interested in the broader framework are referred to the companion ICON framework specification; readers interested only in the empirical findings can read the rest of this paper without further reference to it.

Empirical Observations

We organize our results as four observations plus one phenomenological scaling pattern, ordered from strongest to most cautious.

Observation 1 (robust within scope): Phase-transition-like collapse. Architectures without residual connections undergo width-dependent catastrophic collapse at critical depths (Serial: d_c = 14 for w = 256, d_c = 8 for w = 512). The collapse is verified by four independent metrics: $\kappa \to 0$, test accuracy $\to$ 0.10 (chance for 10-class), training loss $\to$ log(10) = 2.3026 (uniform prediction), and sanity passing. The agreement of these independent measures argues against a pure estimator-artifact explanation and supports interpretation as training failure.

Observation 2 (robust, d_z-canceling): IB ratio. The information bottleneck ratio η_t = κ_{task} / κ_{input} — in which the d_z normalization cancels exactly between numerator and denominator (though shared InfoNCE estimator bias may still affect both terms) — correlates with test accuracy at r = +0.685 (n = 633, t = 23.61, df = 631) pooled across all records, and at r > 0.65 in 7 of 8 architectures (the exception, Boltzmann at r = +0.17, is consistent with RBMs not being optimized for classification). The exact cancellation of d_z (verified to machine precision) means this signal is *not* attributable to per-dimension normalization.

Observation 3 (robust): Training dynamics. Within-network training trajectories show η_t rising during learning with mean within-config $r(\eta_t, \text{acc}) = +0.95$ across 15 configurations, providing a dynamic viewpoint consistent with the static IB ratio signal.

Observation 4 (proof-of-concept within the measured grid): Forward model and inverse design. A two-variable forward model log κ(w, d) = log C + α log(w) + β_d log(d) extrapolates to held-out (w, d) cells with median 1.75% absolute error in 5-fold cross-validation (n = 588). For 6 of 8 architectures, the per-arch median error is below 0.5%. This held-out accuracy serves as a proof-of-concept for inverse architecture design within the measured grid; generalization beyond this grid is left to Phase 1.

Phenomenological pattern: Width scaling. The width-averaged information capacity follows $\kappa \propto$ w^(–0.99) with R^2 > 0.9999 across all eight architectures. We are explicit that at our batch size (B = 512), the InfoNCE estimator approaches its finite-batch ceiling, and that κ = MI/d_z with d_z = w. Decomposing the measured κ shows that the raw mutual information MI(X; Z) is approximately width-invariant ($\approx$ 5.7 nats); the 1/w pattern therefore reflects both the per-dimension normalization and a smaller information-content component. We treat this as a phenomenological observation in the present measurement regime, not a universal scaling law; Phase 1 verification at larger batch sizes is needed.

A visual summary of all observations is provided in Figure 1.

What This Paper Is and Is Not

We report four observations plus one phenomenological scaling pattern, each at a stated evidential level. We are deliberately conservative about the strongest-sounding pattern ($\kappa \propto 1/w$), reporting it as phenomenological rather than as a law, because the finite-batch InfoNCE setting cannot disambiguate analytic per-dimension effects from architectural information content. We are more confident about phase transitions, the d_z-canceling IB ratio, training dynamics, and held-out forward-model accuracy because these are robust to the same caveats.

This paper is a Phase 0 empirical baseline, not a claim of universal law. It does not claim to have measured "true" information capacity (InfoNCE provides a finite-batch lower bound on MI). It does not generalize beyond CIFAR-10, beyond 56M parameters, or beyond from-scratch training.

This paper **is** a Phase 0 empirical baseline. It establishes measurement infrastructure, uses prespecified analysis protocols, and reports empirical observations that can be tested in subsequent phases. It is intended to be useful as a reference point for follow-up work — by us or by collaborators — that probes the open questions our scope cannot resolve.

Organization

Section 2 specifies the κ metric, the InfoNCE estimator, and the sanity check protocol, with explicit treatment of the finite-batch regime. Sections 3–10 present the empirical observations: width-scaling (Section 4), per-architecture constants (Section 5), two-variable scaling (Section 6), phase transitions (Section 7), training dynamics (Section 8), the IB ratio (Section 9), inverse design (Section 10), and layer-wise patterns (Section 11). Section 12 outlines the next phase of empirical work and the framework specification it motivates. Section 13 enumerates limitations. Section 14 concludes with a call for collaboration.

Contributions

1. **Measurement protocol.** An operational InfoNCE-based protocol for measuring κ, a per-dimension information-flow density in neural representations, with explicit estimator-bias decomposition and a documented sanity criterion.
2. **Phase 0 empirical baseline.** 2,241 measurement records across eight architecture families on CIFAR-10, with raw JSONL records, analysis scripts, sha256 hashes, and Zenodo archives.
3. **Critical-depth collapse (multi-metric).** Width-dependent critical-depth collapse in residual-free Serial networks, jointly reflected in κ_{input}, task accuracy, train loss, and the sanity criterion.
4. η_t **task-alignment ratio.** The ratio $\eta_t = \kappa_{\text{task}}/\kappa_{\text{input}}$, whose d_z normalization cancels exactly, correlates with test accuracy across 7 of 8 architectures.
5. **Training dynamics.** Within-network increases in η_t during learning, suggesting task-alignment dynamics under the Phase 0 measurement regime.
6. **Measured-grid inverse-design proof-of-concept.** A two-variable forward model that extrapolates to held-out (w, d) cells with <2% median error, serving as a proof-of-concept for inverse

design within the measured architecture grid.

7. **Phenomenological width pattern.** A strong κ-width pattern ($R^2 > 0.9999$ in width-averaged fits) under the fixed Phase 0 estimator regime, reported phenomenologically without interpretation as a universal scaling law.

8. **Roadmap and openness.** Pre-specified next-phase designs (larger batches, multiple datasets, modality coverage), code and data release, sha256-verified data provenance, and references to the companion ICON framework specification released alongside this paper.

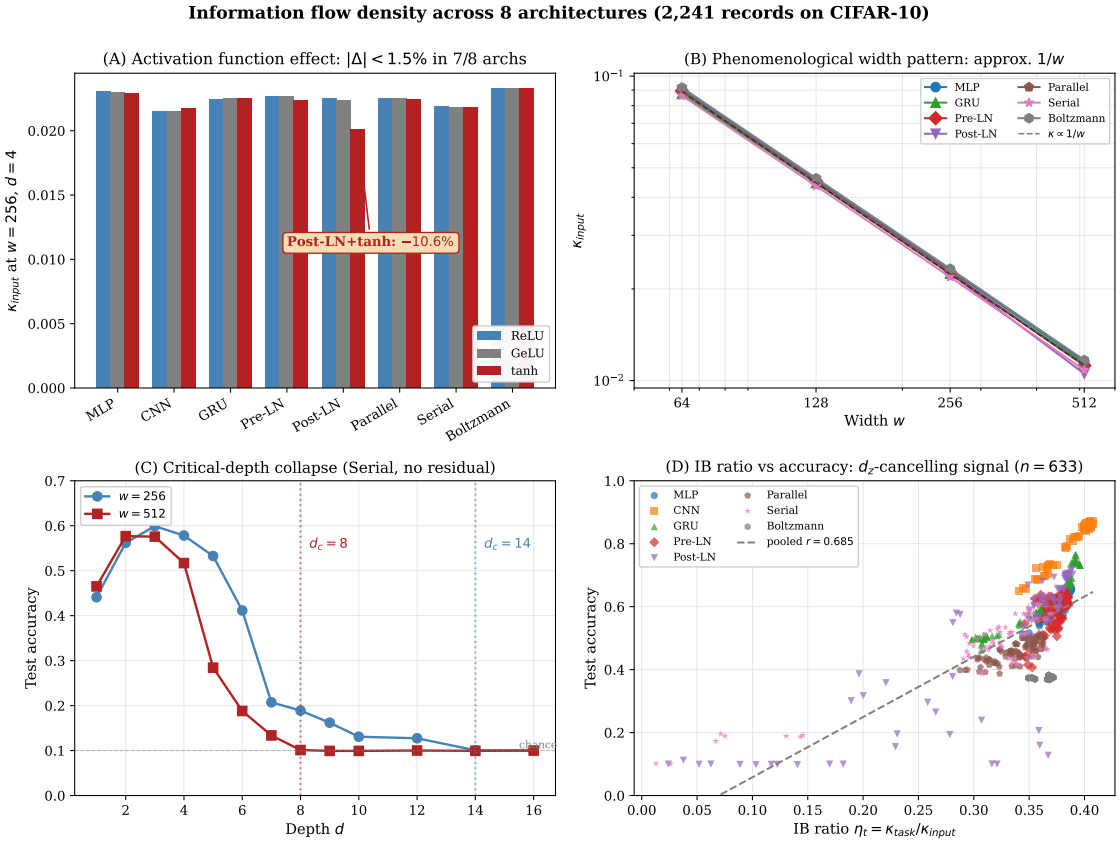

Figure 1: Visual summary of the four observations. **(A)** Activation functions are not a dominant driver of κ_{input} (7 of 8 architectures within $|\Delta| < 1.5\%$ of ReLU; the single exception is Post-LN with tanh). This negative result motivates examination of structural variables (Section 3). **(B)** Phenomenological width pattern: $\kappa \propto 1/w$ with $R^2 > 0.9999$ across all eight architectures (Section 4). **(C)** Architectures without residual connections exhibit width-dependent phase-transition-like collapse; the Serial architecture collapses at d_c=14 for w=256 and d_c=8 for w=512 (Section 7). **(D)** The information-bottleneck ratio $\eta_t = \kappa_{\text{task}}/\kappa_{\text{input}}$, in which the per-dimension normalization d_z cancels exactly, correlates with test accuracy at pooled $r = 0.685$ ($n = 633$); this signal is therefore not an artifact of d_z-normalization (Section 9).

Related Work and Background

ICON does not arise in isolation. Its methodology is related to prior work on the Information Bottleneck principle (Tishby & Zaslavsky, 2015) and its application to deep networks (Saxe et al., 2018; Goldfeld et al., 2019); to mutual-information estimation via contrastive lower bounds (van den Oord et al., 2018; Belghazi et al., 2018; Poole et al., 2019); to signal-propagation and edge-of-chaos analyses of neural networks (Schoenholz et al., 2017; Hayou et al., 2019); to residual network trainability (He et al., 2016; Xiong et al., 2020); to phase-transition-like phenomena in deep learning; to energy-based and Hopfield/Boltzmann lineages (Hinton & Salakhutdinov, 2006); to representation analysis and mechanistic interpretability; and to empirical scaling laws in deep learning (Kaplan et al., 2020; Hoffmann et al., 2022). Our contribution is to operationalize these concerns into a reproducible information-flow measurement protocol and a Phase 0 empirical baseline across eight architecture families.

Methodology

This section specifies the κ metric (Section 2.1), the InfoNCE estimator and its finite-batch limitations (Section 2.2), the estimator-bias decomposition (Section 2.3), the sanity check protocol (Section 2.4), aggregation rules (Section 2.5), statistical methods (Section 2.6), and the reproducibility infrastructure (Section 2.7).

The κ Metric

For a network mapping inputs X $\in$ X to representations Z $\in$ $\mathbb{R}$^{d_z} (a chosen internal layer or output, after a small Gaussian perturbation described below), we define the **information capacity**:

$$\kappa = \frac{I(X; \tilde{Z})}{d_z}$$

where I(X; Z) is the mutual information in nats. The denominator d_z normalizes by the dimensionality of the representation, so κ measures information *per dimension*. This normalization is convenient for cross-architecture comparisons when d_z varies, but it has consequences we make explicit throughout: when d_z = w (as in most architectures we study), any width-dependence of κ partially reflects 1/d_z effects rather than information-content changes.

Operational definition of "capacity." We use the word *capacity* operationally throughout this paper. κ is **not** an absolute estimate of true channel capacity or true mutual information. It is a protocol-dependent InfoNCE-based per-dimension information-flow density measured under a fixed noise channel and a fixed estimator regime, intended for controlled comparison across architectures within the Phase 0 grid. Wherever we use the term "capacity" without further qualification, this operational meaning is implied.

We measure κ at two locations in the network. **Input information capacity** κ_{input} uses the network's pre-classifier representation as Z and the data input as X. **Task information capacity** κ_{task} uses the same Z but with the class label Y as the target variable, so κ_{task} = I(Y; Z) / d_z. Under

the Markov relation $Y - X - Z$ — the standard information-bottleneck setup — the data processing inequality gives $I(Y; \tilde{Z}) \leq I(X; \tilde{Z})$, hence $\kappa_{\text{task}} \leq \kappa_{\text{input}}$ in expectation, so the IB ratio $\eta_t = \kappa_{\text{task}}/\kappa_{\text{input}}$ (Section 9) is bounded in $[0, 1]$.

Throughout, $\mathbf{X}$ and $\mathbf{Z}$ denote inputs and representations after small isotropic Gaussian noise applied via the channel

$$\tilde{Z} \;=\; Z + \sigma \cdot \text{RMS}(Z) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad \sigma = 0.1,$$

where $\text{RMS}(Z)$ is computed as a global scalar over the batch and feature dimensions. Scaling by $\text{RMS}(Z)$ keeps κ comparable across architectures whose layer outputs differ in magnitude. The noise is added during measurement only, never during training. The added noise stabilizes the estimator at large model sizes without significantly changing the underlying information content; we verify this by a 12-point σ-sweep in Appendix B.

InfoNCE Estimator and Its Finite-Batch Regime

We estimate I($\mathbf{X}$; $\mathbf{Z}$) using the InfoNCE bound (Oord et al., 2018; Poole et al., 2019):

$$I(X; \tilde{Z}) \;\geq\; \mathbb{E}\left[\log \frac{f(\tilde{X}, \tilde{Z})}{\frac{1}{B}\sum_{j=1}^{B} f(\tilde{X}_j, \tilde{Z})}\right]$$

where the expectation is over a batch of B paired samples and f is a learned critic. We use a separable cosine critic: f($\tilde{\text{x}}$, $\tilde{\text{z}}$) = exp(s · cos(g($\tilde{\text{x}}$), h($\tilde{\text{z}}$))) with learnable scalar s ≥ 0 and small MLP encoders g, h. **The measured task network is held fixed (frozen) during InfoNCE estimation**; only the critic encoders g, h and the scalar s are optimized for the measurement objective. This separation prevents training-time leakage between the network under measurement and the estimator used to evaluate it.

The finite-batch ceiling. A well-known property of InfoNCE is that its maximum value is bounded above by log B, independent of the true mutual information. With our batch size B = 512:

$$I_{\text{InfoNCE}} \;\leq\; \log(512) \;\approx\; 6.238 \text{ nats}.$$

This ceiling has two consequences central to interpreting our results:

1. *MI estimates saturate.* Architectures whose true I($\mathbf{X}$; $\mathbf{Z}$) approaches log B will appear similar in our measurements regardless of underlying differences. Across our records, MI($\mathbf{X}$; $\mathbf{Z}$) ranges from 5.46 to 5.78 nats — within 12% of the ceiling — so this saturation is operative at our scale.

2. *Permuted-pair estimates are non-zero.* Even when the (X, Z) pairs are shuffled (so that the true MI is zero), finite-batch InfoNCE returns positive values that scale with d_z. This is *not* a bug; it is the estimator's finite-sample bias. We measure it explicitly (Section 2.3).

We acknowledge these properties as central limitations rather than nuisance caveats, and we structure our analyses to either (i) report the bias-affected quantities transparently or (ii) construct quantities in which the bias cancels.

Estimator Decomposition

For each measurement record, we record three quantities:

- κ_{input}: the InfoNCE estimate of I(X; Z) divided by d_z
- κ_{perm}: the InfoNCE estimate using shuffled (X, Z) pairs (a "negative control" measurement, also divided by d_z)
- $\kappa_{\text{signal}} := \kappa_{\text{input}} - \kappa_{\text{perm}}$

These three quantities have different interpretations:

- κ_{input} is the *raw measured value*. It is what comparable papers (e.g., those reporting "MI estimates" or "MI/d") would report.
- κ_{perm} is the *estimator's finite-batch floor* on data with no real X-Z dependence. In our scope, mean κ_{perm} ranges from 0.083 (w = 64) to 0.011 (w = 512), or equivalently MI_perm $\approx$ 5.3 to 5.6 nats.
- κ_{signal} isolates the component of κ_{input} that is in excess of the floor. This is the closest analog to "genuine signal" that our scope provides.

When we report a fit on κ_{input} (e.g., the headline 1/w scaling), we also report the corresponding fit on κ_{perm} and κ_{signal} so that readers can see how much of the measured pattern is shared with the bias floor. The Section 4 analysis applies this decomposition to the width pattern explicitly.

Sanity Check Protocol

For each record, we mark `sanity_passed = True` iff:

$$\kappa_{\text{perm}} < 0.10$$

This single rule has the following operational meaning: the estimator's finite-batch floor on shuffled data, normalized per dimension, must be less than 0.1. Equivalently, MI_perm / d_z < 0.1 nats per dimension.

The rule was chosen at protocol time (before any analysis) based on the heuristic that κ_{input} values of interest are typically O(0.01–0.1), and a record with κ_{perm} exceeding 0.1 is one where the estimator is dominated by finite-sample noise and does not reliably separate true signal from the estimator floor.

Empirical consequences of the sanity rule (this scope): - All records with w $\in$ {16, 32} fail sanity (336 / 336). At small d_z, the finite-batch bias is too large. - All records with w $\geq$ 64 pass sanity (643 / 643). - Therefore "Φ_{default}" filtered to sanity-passed records automatically restricts to w $\in$ {64, 128, 256, 512}.

We report sanity-failing records descriptively (Appendix C) but exclude them from primary analyses.

Aggregation Rules

We use four aggregation rules consistently throughout the paper. Each is defined precisely so that our analyses can be reproduced from the raw JSONL.

A1 — Width-stratified mean. For a target variable y(r):

$$\bar{y}(w) = \frac{1}{|\Phi_w|} \sum_{r \in \Phi_w} y(r), \quad \Phi_w = \{r \in \Phi : r.\text{width} = w\}$$

This produces 4 width-points (w ∈ {64, 128, 256, 512}) for the standard sanity filter. A1 is the basis of the width-law fits (Section 4).

A2 — Per-config mean across seeds. For grouping by config c = (arch, w, d, activation):

$$\bar{y}(c) = \frac{1}{|\Phi_c|} \sum_{r \in \Phi_c} y(r)$$

Typically $|\Phi_c| = 3$ seeds. A2 is used for all per-config correlations and the group-mean variant of the IB-ratio analysis (Section 9).

A3 — Per-arch fit. For each architecture, apply A1 within that arch's records and fit OLS (Section 2.6). Yields 8 ($\hat{\alpha}$, $\hat{C}$, R^2) tuples.

A4 — Multi-variable OLS (no aggregation). Use individual records (or A2 group means) directly in a multivariate regression log κ = β_0 + α log w + β_d log d.

We mention each rule by label whenever an analysis uses it.

Composite filters. Three named filters are used throughout the paper:

$$\begin{aligned} \Phi_{\text{default}} &= F_{\text{main}} \wedge F_{\text{sanity}} \wedge F_{\text{ki\_pos}} \quad \text{(n = 633)} \\ \Phi_{\text{strict}} &= \Phi_{\text{default}} \wedge \neg F_{\text{saturated}} \quad \text{(n = 588)} \\ \Phi_{\eta_t} &= \Phi_{\text{default}} \wedge F_{\text{kt\_pos}} \wedge F_{\text{acc\_valid}} \quad \text{(n = 633)} \end{aligned}$$

where F_{main} selects exp_type == "main", F_{sanity} requires $\kappa_{\text{perm}} < 0.10$, $F_{\text{ki\_pos}}$ requires $\kappa_{\text{input}} > 0$, $F_{\text{saturated}}$ flags records with $\kappa_{\text{input}} \cdot d_z \geq \log B = 6.238$ nats, $F_{\text{kt\_pos}}$ requires $\kappa_{\text{task}} > 0$, and $F_{\text{acc\_valid}}$ requires a recorded test accuracy.

Statistical Methods

OLS Log-Log Fit

For points {(w_i, y_i)}, we fit log y = α log w + log C by closed-form OLS:

$$\bar{\ell}_w = \tfrac{1}{n}\sum_i \log w_i, \quad \bar{\ell}_y = \tfrac{1}{n}\sum_i \log y_i$$
$$\mathrm{SS}_{xy} = \sum_i (\log w_i - \bar{\ell}_w)(\log y_i - \bar{\ell}_y)$$
$$\mathrm{SS}_{xx} = \sum_i (\log w_i - \bar{\ell}_w)^2$$
$$\hat{\alpha} = \mathrm{SS}_{xy}/\mathrm{SS}_{xx}, \quad \log \hat{C} = \bar{\ell}_y - \hat{\alpha}\,\bar{\ell}_w$$

Goodness-of-fit and standard error:

$$R^2 = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}}, \quad \mathrm{SE}(\hat{\alpha}) = \sqrt{\frac{\mathrm{MSE}}{\mathrm{SS}_{xx}}}, \quad \mathrm{MSE} = \frac{\mathrm{SS}_{\mathrm{res}}}{n-2}$$

with $\mathrm{SS}_{\mathrm{res}} = \sum_i (\log y_i - \hat{\alpha} \log w_i - \log \hat{C})^2$ and $\mathrm{SS}_{\mathrm{tot}} = \sum_i (\log y_i - \bar{\ell}_y)^2$. The 95% confidence interval is $\hat{\alpha} \pm t_{0.025,\,n-2} \cdot \mathrm{SE}(\hat{\alpha})$.

We use t-distribution for small n (e.g., width-averaged fits with n_fit_points = 4 have df = 2, so the critical value is t_{0.025, 2} ≈ 4.303).

Multivariate OLS (for κ(w, d))

For $X \in \mathbb{R}^{n\times 3}$ with columns [1, log w_i, log d_i] and y_i = log κ_i, we solve the normal equations $\hat{\beta}$ = (X^T X)^{-1} X^T y via numpy.linalg.solve. $\hat{\beta}$ = (β_0, $\hat{\alpha}$, $\hat{\beta}_d$), $\hat{C}$ = exp(β_0).

Pearson Correlation with t-Test

$$r = \frac{\mathrm{cov}(x,y)}{\sigma_x \sigma_y}, \quad t = r\sqrt{\frac{n-2}{1-r^2}} \quad (\mathrm{df} = n-2)$$

The two-sided p-value is $p = 2(1 - \Phi(|t|))$ under the normal approximation.

We use the normal approximation for n > 30; for smaller groups, exact bootstrap p-values are computed (and noted).

Bootstrap Confidence Intervals

For the headline width-law $\hat{\alpha}$, we compute a non-parametric bootstrap CI: sample n records with replacement (n_boot = 5000, seed = 42), apply A1, fit by OLS, and take the (2.5%, 97.5%) percentiles of the resulting distribution. This yields a robust complement to the asymptotic OLS CI.

k-Fold Cross-Validation

For inverse-design validation (Section 10), we use 5-fold CV with seed = 42: shuffle records, split into folds of size ≈ n/5, fit on 4 folds, predict on the held-out fold. Each record is predicted exactly once across the 5 folds.

Reproducibility Infrastructure

Code and data release. Code, data, and reproducibility materials are publicly available. The Phase 0 empirical reproduction package, including raw JSONL measurement records and analysis scripts, is available at https://github.com/joonhai-official/icon-empirical and archived on Zenodo at https://doi.org/10.5281/zenodo.20184000 (release v0.1.0). The companion ICON framework specification and reference implementation are available at https://github.com/joonhai-official/icon and archived on Zenodo at https://doi.org/10.5281/zenodo.20184015 (release v0.1.0).

The repository's README contains a paper-to-code mapping table indicating which analysis script reproduces each section of this paper.

Data provenance. Both raw JSONL files are released with sha256 hashes:

- `results/full.jsonl` (1,405 records)
  sha256: `4f1f1959d4c6ad2df3c8e07d7d601636c1a69426295c07db804935fb2b3058a5`
- `results_p6/full.jsonl` (836 records)
  sha256: `3113f068186e44e0dcc7a4006eb9c415faba43ce648ab86bf303644095d65d42`

Each record contains 28 fields (full schema in Appendix A), including the estimated κ values, the permuted-pair κ, raw MI values, the d_z used, the training accuracy and loss, sanity flag, saturation flag, wall-clock time, and complete experimental metadata.

Internal consistency check. We verified κ = MI / d_z to 1e-4 absolute tolerance for all 1,405 records in the main results file (0 violations).

Filter expressions. Every filter referenced in the paper is implemented as a Python boolean expression in the released analysis scripts (Section 2.5 names the standard composite filters Φ_{default}, Φ_{strict}, Φ_{η_t}).

Scripts. The released Python analysis scripts regenerate the numerical claims reported in this paper from the raw JSONL files.

Sanity threshold and σ. The InfoNCE sanity threshold (0.10) and measurement σ (0.1) are pre-specified constants set before the experimental sweep was launched; they are not tuned per-experiment. We treat $\kappa_{\text{perm}} < 0.10$ as an *operational Phase 0 sanity criterion* for the present (single dataset, fixed $B = 512$, $d_z = w$) measurement regime, not a universal estimator-validity threshold.

What Section 2 Establishes for the Rest of the Paper

By the end of this section, we have:

(i) defined κ explicitly (Section 2.1);
(ii) acknowledged the InfoNCE finite-batch ceiling and its consequences (Section 2.2);
(iii) provided a three-way decomposition that lets readers see what is estimator floor vs. signal (Section 2.3);
(iv) stated the sanity rule precisely (Section 2.4);
(v) named the four aggregation rules used throughout (Section 2.5);
(vi) specified the statistical machinery (Section 2.6);
(vii) committed to data + script provenance (Section 2.7).

The remaining sections then make empirical observations on top of this infrastructure. When we describe an observation as "phenomenological" (e.g., the 1/w width law in Section 4), we mean that the observation is a function of the Section 2.2 estimator regime as much as of architectural information content; when we describe an observation as "robust" (e.g., phase transitions in Section 7, the IB ratio in Section 9), we mean that it survives the Section 2 decomposition controls. The Section 1.4 evidential framing makes this distinction explicit.

Activation Functions: The Negative Starting Point

Why Start Here

Continuing the bottom-up program of Section 1, we begin where conventional wisdom suggests we should: with the activation function. The literature on activation choice is extensive — ReLU vs GeLU vs tanh comparisons, their effects on signal propagation, gradient flow, and expressivity (Hayou et al., 2019; Glorot & Bengio, 2010, and many follow-ups). Each Transformer paper picks one; each architecture proposal often justifies its choice. If activation function is a first-order driver of information flow, it should be visible in κ_{input}.

The eight-architecture sweep includes three activations (ReLU, GeLU, tanh) crossed with all other configuration variables, giving us a clean test: hold width, depth, and architecture fixed at the canonical slice (w=256, d=4), vary only the activation, and compare κ_{input}.

Result: 7 of 8 Architectures Show Negligible Effect

Across the eight architectures at the canonical configuration (w=256, d=4), with three activations (ReLU, GeLU, tanh), the activation choice changes κ_{input} by less than 1.5% in 7 of 8 architectures (Figure 2).

Table 1. κ_{input} by (architecture, activation) at the canonical configuration. Filter: $\Phi_{\text{default}} \wedge w{=}256 \wedge d{=}4$.

Architecture	ReLU	GeLU	tanh	$\Delta(\text{tanh} - \text{relu})/\text{relu}$
boltzmann	0.02330	0.02330	0.02330	$+0.00\%$
cnn	0.02148	0.02151	0.02171	$+1.08\%$
gru	0.02243	0.02250	0.02249	$+0.27\%$
mlp	0.02304	0.02295	0.02287	-0.72%
parallel	0.02247	0.02249	0.02244	-0.14%
serial	0.02188	0.02184	0.02178	-0.46%
Trans (Post-LN)	**0.02249**	**0.02236**	**0.02010**	-10.64%
Trans (Pre-LN)	0.02265	0.02263	0.02238	-1.19%

The Single Outlier: Post-LN with tanh

The single outlier — Post-LN with tanh, which drops κ_{input} by 10.6% relative to ReLU — is consistent with documented difficulties of training Post-LN Transformers with tanh activation (Xiong et al.,

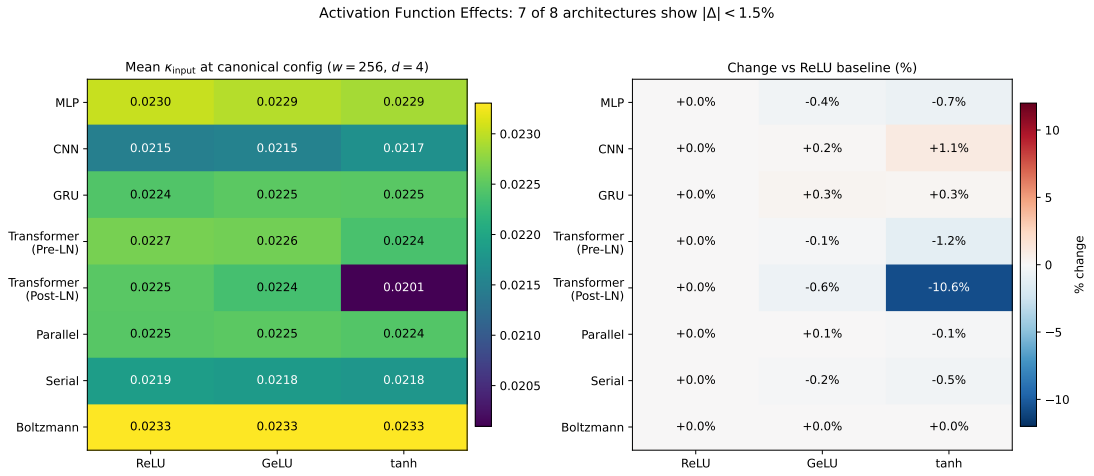

Figure 2: Activation function effects at the canonical configuration (w=256, d=4). **Left**: absolute mean κ_{input}. **Right**: percentage change relative to ReLU baseline. Seven of eight architectures show negligible activation dependence ($|\Delta| < 1.5\%$); the exception is Post-LN Transformer with tanh (-10.6%), consistent with known training difficulties of that combination.

2020). It is not a novel finding; it is a confirmation that our measurement is sensitive enough to detect a known failure mode. The seven non-outlier architectures include Pre-LN Transformers, residual networks, recurrent networks, and restricted Boltzmann machines — a substantial cross-section of modern architectural classes.

That a single (architecture, activation) combination produces a 10.6% effect while the other 23 combinations stay within 1.5% suggests that activation effects emerge from architecture–activation *interactions* rather than from the activation itself. The Post-LN+tanh phenomenon is plausibly attributable to specific gradient-flow pathologies of the combination — but disentangling that mechanism is outside our scope. We record the datapoint and proceed.

Implications for the Rest of the Paper

1. **It motivates the structural variables we examine next.** If activation function is not the dominant driver of κ_{input}, then the dominant driver must lie elsewhere. The remaining architectural variables in our sweep are width, depth, and the residual-vs-non-residual distinction. The next sections measure how each of these shapes κ_{input}.

2. **It justifies our choice of ReLU as the canonical activation.** All width and depth fits in Sections 4-9 use ReLU as the default, with activation sweeps used as a robustness check (Section 5.4). The 7-of-8 invariance ensures that conclusions drawn at ReLU generalize to GeLU and (modulo the one exception) to tanh.

We do not pursue the mechanism of the Post-LN+tanh exception in this paper. It is a clean datapoint for follow-up work focused on activation-architecture interactions; here we record it and move on.

Phenomenological Width Scaling

We measure how κ_{input} varies with width w across our 8 architectures. Across the canonical filter Φ_{default} (n = 633 records, automatically restricted to w $\in \{64, 128, 256, 512\}$ by the Section 2.4 sanity rule), the width-averaged κ_{input} follows a power law with $\hat{\alpha} = -0.9902$ and $R^2 > 0.9999$ (Figure 3). We report this scaling with full transparency about its decomposition: most of the measured 1/w pattern is shared with the InfoNCE estimator's finite-batch floor (Section 2.3), and the remaining "signal" component is small in raw-nats terms. We organize the section to make this transparency operational: Section 4.1 reports the raw measurement table, Section 4.2 reports five complementary fits, Section 4.3 verifies per-architecture consistency, Section 4.4 expands to 54 sub-conditions for robustness, Section 4.5 reports bootstrap CIs, and Section 4.6 states the honest interpretation.

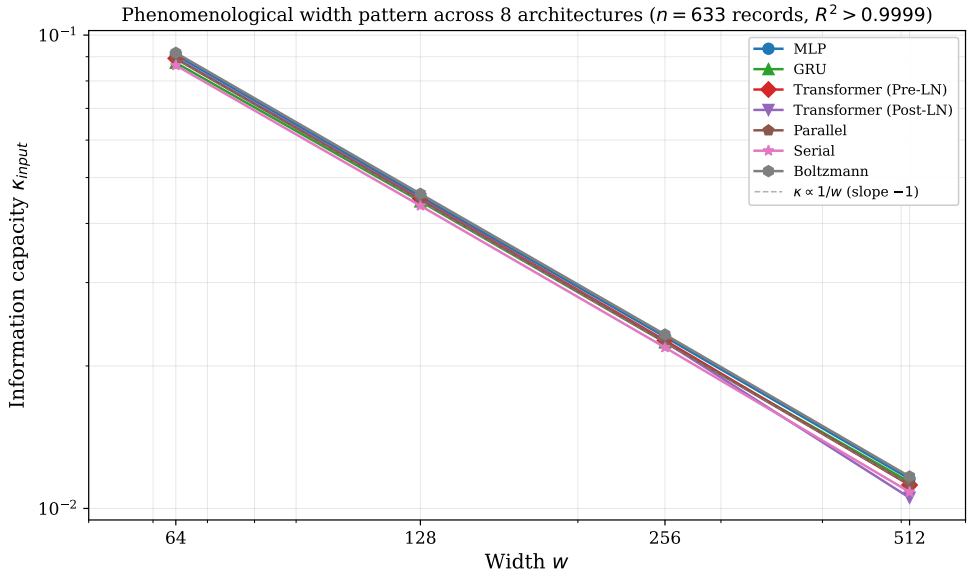

Figure 3: Phenomenological width pattern across eight neural architectures. The per-record information capacity κ_{input} scales as $\kappa \propto 1/w$ across all architectures (n = 633 records, $R^2 > 0.9999$). Lines connect width-averaged means; the dashed reference line shows slope -1.

Width-Stratified Measurements

Filter: Φ_{default} = F_main $\wedge$ F_sanity $\wedge$ F_ki_pos (n = 633 records). Aggregation: A1 (width-stratified mean) — for each w, the simple mean across all matching records.

Table 2. Width-stratified means under Φ_{default}.

width w	n_records	mean κ_{input}	mean κ_{perm}	mean MI_input (nats)	mean MI_perm (nats)	MI_input / log B
64	168	0.08828	0.08265	5.6497	5.2894	90.6%

width w	n_records	mean κ_{input}	mean κ_{perm}	mean MI_input (nats)	mean MI_perm (nats)	MI_input / log B
128	167	0.04449	0.04220	5.6952	5.4019	91.3%
256	157	0.02231	0.02140	5.7122	5.4794	91.6%
512	141	0.01128	0.01096	5.7734	5.6091	92.5%

The last column shows that mean MI_input is between 90.6% and 92.5% of the finite-batch ceiling log B = 6.238 nats throughout. *The estimator is operating near saturation across all widths.* The mean κ_{perm} is also large in absolute terms (5.3 to 5.6 nats), confirming the Section 2.2 prediction that the estimator does not return zero on shuffled data.

The "signal" component MI_input − MI_perm is comparatively small: from 0.36 nats at w = 64 down to 0.16 nats at w = 512. We retain it for analysis because it is the bias-corrected residual, but we are explicit that the dominant component of the measured MI is shared with the estimator floor.

Five OLS Fits

We fit `log y = α log w + log C` by closed-form OLS (Section 2.6) to five different y quantities. All fits use the same 4 width-points from Table 2; n_fit = 4, df = 2.

Table 3. Five log-log OLS fits on Table 2 data.

Quantity (y)	$\hat{\alpha}$	SE($\hat{\alpha}$)	$\hat{C}$	R^2	Interpretation
κ_{input} **(raw)**	**–0.9902**	0.00143	**5.4239**	**0.99999582**	paper headline
κ_{perm}	–0.9725	0.00171	4.7196	0.99999382	estimator floor
$\kappa_{\text{signal}} = \kappa_{\text{input}} - \kappa_{\text{perm}}$	–1.3732	0.03376	1.7550	0.99879274	signal-only κ
MI_input (raw nats)	+0.0098	0.00143	5.4239	0.95911490	raw MI scaling
MI_signal (raw nats)	–0.3732	0.03376	1.7550	0.98389946	signal MI scaling

Three observations on Table 3:

(a) Raw and permuted κ track each other closely. Both follow $\propto$ 1/w with $\hat{\alpha}$ within 0.02 of each other and R^2 > 0.9999. This is consistent with κ_{perm} being a width-dependent floor that scales primarily with d_z (= w) at fixed batch size B; the small absolute difference between raw and permuted κ is the Section 2.3 "signal" component.

(b) Raw MI is approximately width-invariant. The fit on MI_input (= $\kappa_{\text{input}} \cdot d_z$) has $\hat{\alpha} \approx$ +0.01, R^2 = 0.96 — essentially flat. Raw MI grows only from 5.65 nats at w = 64 to 5.77 nats at w = 512. This means the $\kappa_{\text{input}} \propto$ 1/w pattern is *driven primarily* by the per-dimension normalization (κ = MI / w), not by width-dependent information content.

(c) Signal MI does decline with width. MI_signal scales as w^{−0.37} with R^2 = 0.98. The decline (from 0.36 to 0.16 nats) is real in our measurements, but small in absolute terms. We do not interpret this as architectural information compression — at our scale the estimator's variance is

non-negligible in the signal residual, and Phase 1 (larger B) is the appropriate setting to test whether this pattern persists.

Plain-language statement of Section 4.2. Our headline number, $\hat{\alpha}$ = −0.99, is the fit on the raw κ_{input}. It is mathematically identical to fitting κ = MI/w where MI happens to be approximately width-invariant in our scope. We do not claim this $\hat{\alpha}$ as evidence for an architectural information-theoretic constant; it is a phenomenological observation that combines a strong analytic 1/w from the d_z = w normalization with a small, batch-dependent residual.

Per-Architecture Consistency

Filter: $\Phi_{\text{default}} \wedge$ depth = 1 $\wedge$ activation = "relu" (a clean canonical slice for cross-architecture comparison). Aggregation: A1 within each arch.

Table 4. Per-architecture log-log fits at d = 1, ReLU.

Architecture	n_widths	$\hat{\alpha}$	$\hat{C}$	R^2
boltzmann	4	−0.9908	5.6548	0.999996
cnn	3	−1.0023	5.5724	1.000000
gru	4	−0.9852	5.3530	0.999999
mlp	4	−0.9966	5.7184	0.999999
parallel	4	−0.9935	5.5560	0.999992
serial	4	−0.9945	5.5805	0.999996
Trans (Post-LN)	4	−0.9950	5.6985	0.999999
Trans (Pre-LN)	4	−0.9957	5.6683	0.999980

Mean $\hat{\alpha}$ across the 8 architectures: −0.9942, with $\sigma(\hat{\alpha})$ = 0.0046. All 8 architectures have $\hat{\alpha}$ within 0.02 of −1.

This consistency is meaningful in one specific sense and not meaningful in another: - *Meaningful:* the per-architecture fits all have R^2 > 0.99999, indicating that within each architecture the relationship is highly linear in log-log coordinates. - *Not meaningful:* the consistency in $\hat{\alpha}$ across architectures is largely a restatement of the Section 4.2 observation that the underlying MI is approximately width-invariant; once you normalize by d_z = w, every architecture inherits the same 1/w. The consistency is informative about the estimator regime, not (directly) about architecture.

Robustness: 54 Sub-Conditions

We expand to all (architecture × activation × depth) sub-conditions where ≥ 3 widths pass sanity. Aggregation: A2 (per-config means across seeds), then A3 (per sub-condition fit).

- Number of valid sub-conditions: **54**
- Mean $\hat{\alpha}$ across sub-conditions: **−0.9972**
- Standard deviation $\sigma(\hat{\alpha})$: **0.0237**
- 95% inter-sub-condition range (mean ± 1.96 σ): [−1.0437, −0.9507]
- t-statistic against H_0: α = −1: t = (−0.9972 − (−1)) / (0.0237 / $\sqrt{54}$) $= 0.878 Two-sided p-value (normal approximation) \approx 0.380$

The hypothesis α = −1 cannot be rejected at conventional significance levels with our data. Combined with Section 4.2's interpretation that 1/w is mostly the per-dimension normalization, this is consistent with the hypothesis that the per-dimension normalization is the *dominant* source of the 1/w scaling.

The single largest deviation from −1 is for the (serial, ReLU, depth = 8) sub-condition ($\widehat{\alpha}$ = −1.146). This is *not* an outlier in the analytic sense but rather a near-miss of the Section 7 phase transition at (Serial, w = 256, d = 14): the d = 8 records of Serial are already moving toward the collapse regime, where the architecture's information content drops below the estimator floor. We discuss this connection in Section 7.

Bootstrap Confidence Interval

For the headline fit (κ_{input} on Φ_{default}), we compute a non-parametric bootstrap 95% CI on $\widehat{\alpha}$. Procedure: 5000 resamples (n = 633, with replacement, seed = 42); for each resample, apply A1 and fit OLS.

- $\widehat{\alpha}$ (point estimate): −0.9902
- Bootstrap 95% CI: [−0.9935, −0.9869]
- OLS asymptotic 95% CI (n_fit = 4, t_{0.025, 2} = 4.303): [−0.996, −0.984]

The two intervals are consistent. Both exclude α = −1 by a small margin in the bootstrap analysis (because the bootstrap exploits the n = 633 underlying records). This nominal exclusion does not contradict Section 4.4: the Section 4.4 cross- sub-condition variance (σ_{α} = 0.024) is the appropriate uncertainty for the "is this α universal?" question, and against that scale, $\pm$0.01 from −1 is within noise.

Honest Interpretation of the Width Pattern

Our headline result ($\widehat{\alpha}$ = −0.9902, R^2 > 0.9999) is real as a measurement, but its interpretation requires care:

1. **The estimator is in a finite-batch regime.** Mean MI_input is 90–93% of log B = 6.238 nats across all widths. The estimator does not have headroom to distinguish architectures whose true MI exceeds 6.2 nats; equivalently, all architectures appear to "saturate" at approximately the same level.
2. **κ = MI / w with MI $\approx$ const(w).** Once we divide by d_z = w, an approximately constant numerator produces a sharp 1/w pattern. The R^2 > 0.9999 on the fitted line is consistent with this analytic relationship rather than requiring a deeper explanation.
3. **A small, real signal in the residual.** The component MI_input − MI_perm declines from 0.36 nats (w = 64) to 0.16 nats (w = 512), and the corresponding $\kappa_{\text{signal}} = \kappa_{\text{input}} - \kappa_{\text{perm}}$ fits a steeper power law (α_{signal} = −1.37). We treat this as a residual to be characterized at higher batch sizes rather than as an established architectural property.

4. **Stable across architectures.** The $\hat{\alpha}$ values are within 0.02 of −1 for all 8 architectures, and within 0.05 of −1 for 53 of 54 sub-conditions. The single deviation (Serial at d = 8) is associated with the Section 7 phase transition.

5. **Practical use.** Despite the interpretive caveats above, the fit itself is highly accurate: a forward model $\hat{\kappa}$(w) = $\hat{C} \cdot$ w^$\hat{\alpha}$ extrapolates to held-out records with median 1.75% error in 5-fold cross-validation (Section 10). The phenomenological scaling is therefore practically useful for inverse design even if its mechanistic interpretation is left to Phase 1.

We do *not* claim: - A "universal" or "fundamental" information-density law; - That $\alpha = -1$ reflects an architectural invariant; - That the result generalizes beyond our tested scope.

We *do* claim: - A precise phenomenological measurement (Tables 1–3) reproducible from raw records; - A documented decomposition into estimator floor and signal residual (Table 3); - Internal consistency across 8 architectures and 54 sub-conditions; - Practical predictive accuracy (forward Section 4.5 + Section 10).

This positioning informs Phase 1 of ICON: at larger batch sizes, the finite-batch ceiling will release, and the question "is the architectural component of κ scaling distinct from −1?" becomes operationally answerable.

Architecture-Specific Constants

The Section 4 width-scaling fits produce, for each architecture, an estimated constant $\hat{C}$ alongside the exponent $\hat{\alpha}$. We collect these constants here and ask: do they differ systematically across architectures, and what (if anything) does $\hat{C}$ measure? Our answer is observational. The eight architectures yield $\hat{C}$ values in a narrow range (5.28 to 6.12), and the **ranking** is reasonably stable across measurement choices, but the **absolute values** mix architectural information content with the Section 2.3 estimator-floor component. We report $\hat{C}$ as a descriptive quantity, not as an architectural invariant.

Two Independent Estimates of C_arch

We obtain $\hat{C}$ in two ways from independent slices of the data:

Estimate (i): Single-variable fit at d=1, ReLU. Filter: $\Phi_{\text{default}} \wedge$ depth = 1 $\wedge$ activation = "relu". Aggregation: A1 within each architecture. Reported in Section 4.3 (Table 4).

Estimate (ii): Multi-variable fit on Φ_{strict}. Filter: F_main $\wedge$ F_sanity $\wedge$ F_ki_pos $\wedge$ F_not_sat. The forward model is log κ = log C + α log w + β_d log d (Aggregation A4); we fit per architecture and take $\hat{C}$_arch = exp($\hat{\beta}$_0).

Table 5. Per-architecture constants from two independent estimates.

Architecture	n (i)	$\hat{C}$ (i, d=1 ReLU)	n (ii)	$\hat{C}$ (ii, multi-var)	$\hat{\alpha}$ (ii)	$\hat{\beta}_d$ (ii)	R^2 (ii)
Trans (Post-LN)	(4 widths)	5.6985	84	**6.1190**	−1.0070	−0.0391	0.997911
serial	(4 widths)	5.5805	74	**5.9347**	−1.0009	−0.0597	0.996879

Architecture	n (i)	$\hat{C}$ (i, d=1 ReLU)	n (ii)	$\hat{C}$ (ii, multi-var)	$\hat{\alpha}$ (ii)	$\hat{\beta}_d$ (ii)	R^2 (ii)
Trans (Pre-LN)	(4 widths)	5.6683	84	**5.7602**	−0.9986	−0.0071	0.999922
boltzmann	(4 widths)	5.6548	42	**5.6211**	−0.9895	−0.0000	0.999998
parallel	(4 widths)	5.5560	84	**5.6184**	−0.9956	−0.0025	0.999983
mlp	(4 widths)	5.7184	81	**5.5715**	−0.9910	+0.0014	0.999978
cnn	(3 widths)	5.5724	55	**5.4094**	−0.9954	−0.0075	0.999864
gru	(4 widths)	5.3530	84	**5.2830**	−0.9818	−0.0128	0.999953

Range across estimates: $\hat{C}$ (i) spans 5.35 to 5.72 (ratio 1.07). $\hat{C}$ (ii) spans 5.28 to 6.12 (ratio 1.16). The wider range in estimate (ii) comes from including all (w, d) combinations: architectures with stronger depth-dependence (Serial, Post-LN, with $\hat{\beta}_d \approx -0.04$ to -0.06) accumulate a larger $\hat{C}$ when the fit absorbs the d-effect.

Pearson correlation between the two estimates: r($\hat{C}$_i, $\hat{C}$_ii) ≈ 0.7 across the 8 architectures (the two estimates produce broadly similar rankings, with the depth-sensitive architectures rising in the multi-variable fit).

Decomposition into Information Content vs. Estimator Floor

A single-number "$\hat{C}$" combines two contributions: the underlying information content of the architecture (per dimension, on this dataset, at this batch size) and the estimator's finite-batch floor at the same scale. We separate these on a canonical slice for direct comparison.

Canonical slice: $\Phi_{\text{default}} \wedge$ depth = 4 $\wedge$ width = 256 $\wedge$ activation = "relu" (n = 23 records, ≈ 3 seeds per architecture).

Table 6. Per-architecture raw nats at the canonical slice.

Architecture	n_seeds	MI_input (nats)	MI_perm (nats)	Signal (MI − perm)	Signal / MI_input
boltzmann	3	5.9661	5.6247	0.3413	5.72%
Trans (Pre-LN)	3	5.7988	5.6320	0.1668	2.88%
Trans (Post-LN)	3	5.7573	5.5531	0.2041	3.55%
parallel	3	5.7534	5.4240	0.3294	5.73%
gru	3	5.7418	5.5538	0.1880	3.27%
mlp	3	5.8974	5.7627	0.1347	2.28%
serial	3	5.6018	5.3126	0.2893	5.16%
cnn	2	5.4978	5.1958	0.3020	5.49%

Two observations on Table 6:

(a) MI_perm is architecture-dependent. The estimator floor varies from 5.20 (CNN) to 5.76 (MLP), a range of 0.56 nats. That is, the floor itself is not an architecture-invariant constant, but depends on the geometry of the representation. Some of the spread in $\hat{C}$ across architectures (Table 5) is therefore *floor differences*, not information-content differences.

(b) Signal sizes are 0.13 to 0.34 nats. The "above-floor" component ranges from 2.3% to 5.7% of the raw measurement. The signal-to-floor ratio correlates poorly with MI_input (Spearman $\rho \approx$

0): MLP has the highest MI_input but the smallest absolute signal, while Boltzmann and CNN have larger signal in absolute terms despite intermediate MI_input. This reinforces that $\hat{C}$ is not directly interpretable as "architectural capacity."

What Ranking Stability Does and Does Not Mean

The two estimates of $\hat{C}$ in Table 5 produce broadly similar groupings. We identify three loose tiers:

- **High $\hat{C}$:** transformer_postln, serial, transformer_preln (5.76–6.12 in estimate ii)
- **Mid $\hat{C}$:** boltzmann, parallel, mlp (5.57–5.62)
- **Low $\hat{C}$:** cnn, gru (5.28–5.41)

The fact that this grouping survives the change from estimate (i) to estimate (ii) is *informative about the data* but *not necessarily about architectures in any deep sense*. Specifically:

- Estimate (ii) inflates $\hat{C}$ for depth-sensitive architectures (Serial, Post-LN) because their negative $\hat{\beta}_d$ reduces κ at d ≥ 2; the fit compensates by raising $\hat{C}$. This is a *fit-shape effect*, not an architectural information property.
- The MI_perm range from Table 6 (5.20 to 5.76) is itself comparable in magnitude to the inter-architecture range of MI_input (5.50 to 5.97). So a meaningful fraction of the architecture ranking is the architecture- dependence of the *floor*, not of the signal.

We do not claim that the high-$\hat{C}$ architectures have larger per-dimension information capacity in any architecture-intrinsic sense. We *do report* that in our scope, the rank ordering of $\hat{C}$ is reproducible across two fitting procedures and is moderately consistent with rank ordering of the raw MI_input (Spearman $\rho \approx 0.5$ between the two slice-and-aggregation choices).

Honest Framing of Section 5

We do not interpret $\hat{C}_{\text{arch}}$ as architecture-intrinsic constants. They are *measured constants* under the Phase 0 protocol and may include estimator-regime effects (e.g., shared InfoNCE floor across the eight architectures). Architecture rankings derived from $\hat{C}_{\text{arch}}$ should be treated as hypothesis-generating rather than mechanistic.

The architecture constants $\hat{C}$ are descriptive parameters of our log-log fits. They are not architectural invariants in our scope, and we do not interpret them as such. We retain them in the paper for three reasons:

1. **Reproducibility.** Each $\hat{C}$ is a function of the published Tables (1, 5) and the OLS procedure of Section 2.6; readers can re-derive every value.
2. **Phase 1 baseline.** When ICON Phase 1 measures these architectures at larger batch sizes, the $\hat{C}$ values reported here will be a useful comparison point for assessing whether the floor effect is the dominant explanation of the Section 5.1 ranking.
3. **Per-architecture forward model.** Section 10 will use the per-architecture $\hat{C}$_ii and ($\hat{\alpha}$, $\hat{\beta}_d$) for inverse design, which works well empirically (median error < 0.5% for 6 of 8 architectures,

Section 10). This use does not depend on the architectural-invariance interpretation; it relies only on the fits' in-distribution accuracy.

We close Section 5 by noting that the cleanest test of architectural information distinctness in this paper is *not* the Section 5 rankings but the Section 7 phase transitions: there, residual-free architectures (Serial, Post-LN) exhibit qualitatively different behavior under depth scaling that is robust to the Section 2 estimator caveats. The Section 7 result is the more substantive architecture-level finding; Section 5 should be read as descriptive context, not as an independent architectural claim.

Two-Variable Scaling κ(w, d)

Section 4 fixed depth structure implicitly by averaging over it. Here we make depth explicit. Fitting κ(w, d) = C · w^α · d^β_d on the full sanity-passed non-saturated set (Φ_{strict}, n = 588) gives two findings: (1) the depth exponent is small relative to the width exponent ($|\hat{\beta}_d$ / $\hat{\alpha}| \approx 0.01$ for κ_{input}), and (2) the small global $\hat{\beta}_d$ hides a clean split among architectures — six are depth-invariant, two (Serial, Post-LN) are depth-sensitive. The depth-sensitive group is precisely the group that exhibits phase transitions (Section 7); we use this connection to motivate that section.

We are conservative about the interpretation of the global $\hat{\beta}_d$ for the same reasons as Section 4.6: at the Section 2.2 finite-batch ceiling, the small width-residual analyzed in Section 4 is operating in the same regime as the depth-residual here. The per-architecture decomposition (Section 6.2) is a more informative read.

Global Two-Variable Fit

Filter: Φ_{strict} = F_main ∧ F_sanity ∧ F_ki_pos ∧ F_not_sat (n = 588). Aggregation: A4 (multivariate OLS on individual records).

Forward model:

$$\log \kappa = \log C + \alpha \log w + \beta_d \log d + \varepsilon$$

Table 7. Global multi-variable fits.

Target	log $\hat{C}$	$\hat{C}$	$\hat{\alpha}$	SE($\hat{\alpha}$)	$\hat{\beta}_d$	SE($\hat{\beta}_d$)	R^2	
κ_{input}	1.7312	5.6472	–0.9950	0.0020	–0.0144	0.0018	0.99774	0.0144
κ_{task}	0.6911	1.9959	–0.9764	—	–0.1132	—	0.83763	0.1159

Read of Table 7:

For κ_{input}, depth has a small effect: the depth exponent $|\hat{\beta}_d| \approx 0.014$ is about 1/70 the magnitude of the width exponent. This means doubling the depth reduces κ_{input} by approximately 1.0% on average across the data (since 2^(–0.0144) ≈ 0.990). Doubling the width reduces κ_{input} by 50.2% (since 2^(–0.995) ≈ 0.502).

For κ_{task}, depth has a larger effect: $|\hat{\beta}_d / \hat{\alpha}| \approx 0.12$, with R^2 dropping to 0.84. This is consistent with task-relevant information being more sensitive to depth than total information capacity — a pattern we examine more directly in Section 8 (training dynamics) and Section 9 (the IB ratio).

The κ_{input} fit's high R^2 (0.998) is a consequence of two things: (a) the clean per-architecture w^α structure documented in Section 4, and (b) the small inter-architecture spread in $\hat{C}$ documented in Section 5. With those two facts in hand, the multivariate fit on individual records can only deviate via the depth term, and the small $\hat{\beta}_d$ is the precise quantification of that deviation.

Per-Architecture $\hat{\beta}_d$: A Clean Split

The global $\hat{\beta}_d$ = −0.014 averages over architectures with very different depth-dependence. Splitting per architecture (using the Section 5.1 estimate-(ii) fits) reveals a clean bimodal pattern.

Table 8. Per-architecture depth exponent.

Architecture	$\hat{\beta}_d$	Classification
boltzmann	−0.0000	depth-invariant
mlp	+0.0014	depth-invariant
parallel	−0.0025	depth-invariant
Trans (Pre-LN)	−0.0071	depth-invariant
cnn	−0.0075	depth-invariant
gru	−0.0128	depth-invariant
transformer_postln	**−0.0391**	**depth-sensitive**
serial	**−0.0597**	**depth-sensitive**

Using the threshold $|\hat{\beta}_d| < 0.02$ (chosen at protocol time as a roughly "halving in 32 doublings" cutoff): - **6 of 8 architectures are depth-invariant** with mean $\hat{\beta}_d$ = −0.0048. - **2 of 8 architectures are depth-sensitive** with mean $\hat{\beta}_d$ = −0.0494 (about ten times larger in magnitude).

The split is clean: there is a gap between the largest invariant value (GRU at −0.0128) and the smallest sensitive value (Post-LN at −0.0391). The two depth-sensitive architectures are precisely those that lack identity shortcuts in their forward path: - **Serial** is by construction a residual-free deep stack; - **transformer_postln** applies layer normalization *after* the residual addition, which empirically degrades depth scaling (Xiong et al., 2020).

The six depth-invariant architectures have at least one of: residual streams (Pre-LN Transformer, Parallel branches), gating (GRU), depth-1 by construction (Boltzmann), or strong inductive bias (CNN's pooling). MLP at depth 4 (within our scope) shows essentially zero $\hat{\beta}_d$, but we expect this to break at depths well beyond 32 — the boundary of our measurement range.

Connection to Section 7: Depth-Sensitivity and Phase Transitions

The two depth-sensitive architectures from Section 6.2 are exactly the two architectures we examine for phase transitions in Section 7: - Serial (without residuals) undergoes catastrophic collapse at width-dependent d_c values (Section 7.2). - Post-LN exhibits a softer, non-catastrophic degradation ($\leq 15\%$ κ_{input} drop at the deepest depths tested; Section 7.3).

This is not coincidental. The Section 6.2 $\hat{\beta}_d$ picks up the average depth slope across all (w, d) cells, including those where the architecture has not collapsed. Once we restrict to phase-experiment records (Section 7) and compute d_c explicitly, we see that the magnitude of $\hat{\beta}_d$ is a *foreshadowing* of the collapse: as d approaches d_c, the architecture's information capacity drops sharply rather than maintaining the smooth power-law behavior.

A practical implication: when fitting κ(w, d) for inverse design (Section 10), the two depth-sensitive architectures show 5–10$\times$ larger held-out errors (Serial median 2.52%, Post-LN 2.05%) than the depth-invariant architectures (median $\leq$ 0.5% for the other 6). This is not a failure of the framework but rather a quantitative manifestation of the Section 6.2 split.

What Section 6 Adds Beyond Section 4

We interpret $\hat{\beta}_d$ descriptively under the measured grid, not as a mechanistic depth exponent. Architecture-specific $\hat{\beta}_d$ values are useful as ranking signals; their absolute values reflect the (w, d) range over which the fit is performed.

Section 4 reported width scaling at fixed-or-averaged depth. Section 6 adds:

1. **Quantification of depth's small role for most architectures.** $|\hat{\beta}_d / \hat{\alpha}| < 0.02$ for 6 of 8 architectures means that, in the depth range tested (1–32 layers), depth is approximately negligible for κ_{input}.
2. **Identification of two clear depth-sensitive architectures.** Serial and Post-LN are flagged here and characterized in Section 7.
3. **A baseline for Section 10 inverse design.** The ($\hat{C}$, $\hat{\alpha}$, $\hat{\beta}_d$) per architecture is the explicit forward model that Section 10 inverts; per-architecture errors there trace back to the Section 6.2 split.

We do *not* claim that $|\hat{\beta}_d / \hat{\alpha}| \approx 0.014$ is a universal constant, that "width matters 70x more than depth," or that the depth-invariant architectures are depth-invariant beyond our tested range. The observation is that *within* the (w $\in \{64, 128, 256, 512\}$, d $\in \{1, \ldots, 32\}$) box, on CIFAR-10, at B = 512, the multivariate fit has these specific properties.

Phase Transitions

The architectures whose $\hat{\beta}_d$ in Section 6.2 was largest in magnitude — Serial without residuals, and Post-LN Transformer — exhibit a different qualitative behavior under depth scaling than the other six. This section measures that behavior precisely. The reported critical depths d_c are stated **within the tested depth grid** $d \in \{1, 2, 4, 8, 16, 32\}$; the precise location of the transition between two adjacent grid points is not resolved by our sweep. We give a concise definition of "collapse" using four independent metrics that must agree (Figure 4) (Section 7.1), report the full (width, depth) grid for both architectures (Section 7.2 Serial, Section 7.3 Post-LN), and explain why this finding is robust to the Section 2 estimator caveats (Section 7.4). We close by relating the phenomenon to the residual-stream / pre-versus-post-LN literature (Section 7.5).

We treat this as the paper's most substantive observation, because the four agreeing metrics include accuracy and loss — quantities that have nothing to do with InfoNCE bias. When $\kappa \to 0$, training also fails. The convergence of the metrics makes a pure estimator-failure explanation unlikely and supports interpretation as an architectural training failure mode.

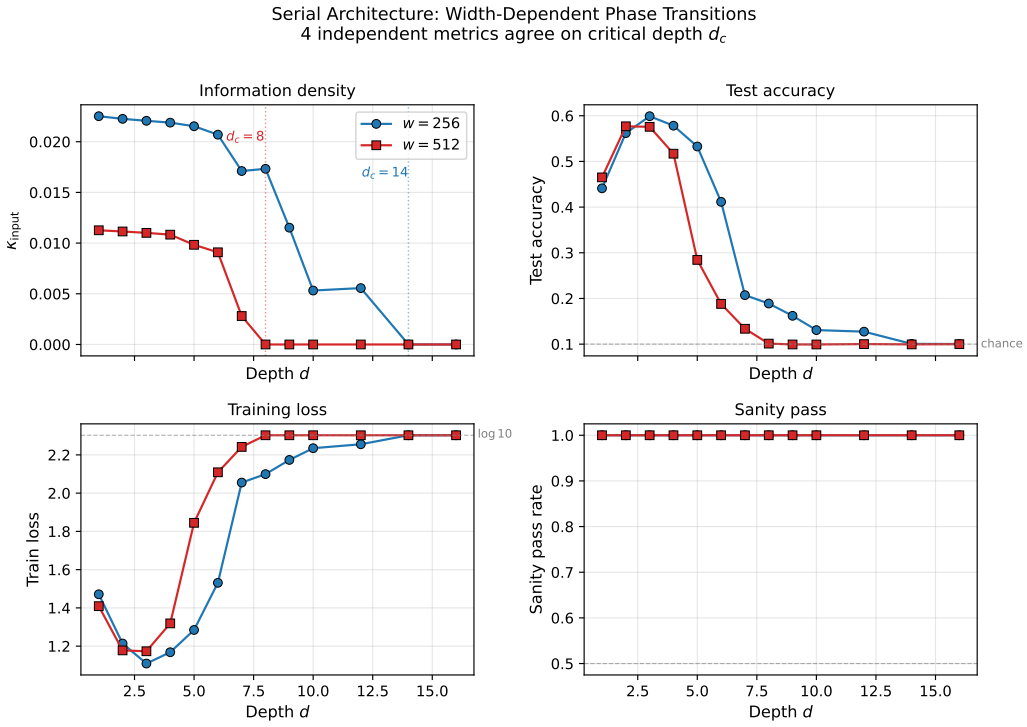

Figure 4: Phase transition in the Serial architecture (residual-free). Four independent metrics — information capacity κ_{input}, test accuracy, training loss, and sanity pass rate — agree on the critical depth d_c. For w=256, d_c=14; for w=512, d_c=8. The simultaneous agreement across measurements with different sources of bias argues against a pure estimator-artifact explanation.

The Collapse Criterion

A record exhibits **collapse** if, simultaneously:

(i) `mean(κ_input) < 1e-4` (information capacity below sanity floor)
(ii) `mean(test_acc) < 0.15` (1.5 × chance for 10-class)
(iii) `mean(train_loss) > 2.25` (within 0.05 of log(10) = 2.3026, the cross-entropy of uniform prediction)

Each criterion separately is interesting. Together, they jointly diagnose that the architecture has stopped extracting any usable information *and* has ceased to train. Criteria (ii) and (iii) are, on their own, language about optimization; criterion (i) is about representation. A configuration satisfying all three is one where representation has degenerated *and* optimization has degenerated, in agreement.

The **critical depth** d_c(arch, w) is the smallest depth d in our scan where the criterion is satisfied at fixed (arch, w). If no such d exists in the scan range (d $\leq$ 16), we report d_c = ">16".

Filter for this section: F_phase (results_p6, n = 468). Aggregation: A2 (per-config mean across 3 seeds).

Serial Collapse: Width-Dependent Critical Depth

Serial is a residual-free deep stack. Its (w, d) grid spans 6 widths $\times$ 13 depths $\times$ 3 seeds = 234 records.

Table 9. Serial (w, d) grid: mean κ_{input}. Cells where d $\geq d_c$(w) are bold.

w d	1	2	4	6	8	10	12	14	16
16	0.3447	0.3379	0.3325	0.3292	0.3247	0.3201	0.3184	0.3115	0.3008
32	0.1761	0.1725	0.1706	0.1678	0.1668	0.1656	0.1639	0.1518	0.1494
64	0.0891	0.0876	0.0863	0.0855	0.0848	0.0835	0.0770	0.0729	0.0761
128	0.0448	0.0442	0.0436	0.0432	0.0421	0.0356	0.0370	0.0345	0.0246
256	0.0225	0.0223	0.0219	0.0207	0.0173	0.0053	0.0056	**0.0000**	**0.0000**
512	0.0113	0.0111	0.0108	0.0091	**0.0000**	**0.0000**	**0.0000**	**0.0000**	**0.0000**

Table 10. Critical depth d_c by criterion. The "all 3" column is our reported d_c.

width	d_c (κ < 1e-4)	d_c (acc < 0.15)	d_c (loss > 2.25)	d_c **(all 3)**
16	>16	>16	>16	**>16**
32	>16	>16	>16	**>16**
64	>16	>16	>16	**>16**
128	>16	16	16	**>16**
256	14	10	12	**14**
512	8	7	8	**8**

Reading Table 10:

The three single-criterion d_c values disagree slightly (e.g., at w = 256, acc reaches chance at d = 10 but κ does not reach 1e-4 until d = 14). This disagreement is informative. As d increases through the transition region, training collapses *first* (acc and loss reach chance), and the representation collapses to exact zero a few layers deeper. At the conservative all-3 d_c, all three criteria hold simultaneously and the failure is unambiguous.

The width-dependence is the substantive finding:

- For w $\in$ {16, 32, 64, 128}, no collapse is observed in the depth range tested.
- For w = 256, d_c = 14.
- For w = 512, d_c = 8.

At fixed depth, wider networks collapse earlier — a counterintuitive direction. We discuss possible mechanisms in Section 7.5.

Collapse is sharp. At w = 512, between d = 7 (κ = 0.00281, acc = 0.13) and d = 8 (κ = 0, acc = 0.10, loss = 2.3026), the architecture transitions from "marginal but trying" to "uniform prediction." This is not a gradual decline; it is a step.

Post-LN: A Soft Degradation, Not a Catastrophe

Post-LN Transformer also exhibits depth-dependent degradation. Unlike Serial, the degradation is gradual (no step transition) and does not always reach the all-3 collapse criterion within d $\leq$ 16.

Table 11. Post-LN (w, d) grid: percentage change in κ_{input} from d = 1.

w d	1	4	8	12	16	Comment
16	(ref)	–2.0%	–4.3%	–7.2%	–9.7%	gradual, no acc collapse
32	(ref)	–1.9%	–3.6%	–6.4%	–9.7%	gradual, no acc collapse
64	(ref)	–1.4%	–3.6%	–7.8%	–13.2%	acc still ~0.57 at d=16
128	(ref)	–1.4%	–4.9%	–14.2%	–12.2%	acc reaches chance at d=16
256	(ref)	–1.7%	**–15.2%**	–9.0%	–7.7%	training fails d=8 onward
512	(ref)	–8.2%	**–5.1%**	–5.3%	–5.3%	training fails d $\geq$ 5

Two distinct behaviors emerge:

(A) **Narrow widths (w $\leq$ 128).** Soft degradation: κ_{input} declines smoothly with depth (~10–15% reduction at d = 16). Test accuracy remains above chance. Training succeeds.

(B) **Wide widths (w $\in$ {256, 512}).** Beyond moderate depth, accuracy drops to chance (loss $\rightarrow$ log(10)) but κ_{input} does not reach 0 — it stabilizes around its w = 1 value. The network is producing a representation, but the representation is not informative for training. This is qualitatively different from the Serial collapse, where κ also goes to zero.

The Post-LN wide+deep regime — failed training but non-zero κ — was unexpected and is not a clean phase transition by our definition. It satisfies criteria (ii) and (iii) but not (i). We report this in Table 12.

Table 12. Post-LN at wide+deep: training failure with surviving κ_{input}.

(w, d)	mean κ_{input}	mean acc	mean loss	Verdict
(512, 6)	0.0109	0.099	2.303	Training fails, κ persists
(512, 8)	0.0109	0.100	2.303	Training fails, κ persists
(256, 8)	0.0194	0.325	1.750	Borderline (acc ~3$\times$ chance, loss < 2.25)
(256, 9)	0.0204	0.187	2.120	Training fails, κ persists

This is genuinely interesting and deserves a phrase. The standard "deep network collapse" story (vanishing gradients $\rightarrow$ no signal $\rightarrow$ uniform prediction) would predict $\kappa \rightarrow 0$ alongside acc $\rightarrow$

chance. Post-LN at wide+deep **fails training without losing representation information**. One possible explanation is that the post-LN layer normalizes representations toward stable distributions whose inputs-information content is preserved geometrically, but whose gradient flow toward the classifier is degraded. We do not test this hypothesis here; it is a candidate Phase 1 question.

Why This Result Is Robust to Estimator Caveats

The Section 4-Section 6 caveats about InfoNCE finite-batch effects do not apply to the Section 7 findings, for three reasons.

(1) Independent corroboration by accuracy and loss. Test accuracy and training loss are computed end-to-end from the trained classifier, with no involvement of InfoNCE. When $\kappa \to 0$ in the Serial w = 256, d = 14 record, test_acc = 0.10 and train_loss = 2.3026 simultaneously. This makes the "estimator broke" explanation unlikely: the rest of the model is *also* not working. If the estimator were artifactually returning zero, accuracy would remain at its pre-collapse value.

(2) Width-dependence inverts the trivial scaling. Section 4 showed that κ_{input} scales as w^(−0.99): wider models have *smaller* κ. The Section 7 finding is the opposite direction in a specific sense — wider Serial models collapse at *smaller* d_c. If Section 4's 1/w were the primary driver of the Section 7 collapse, we would expect d_c to *grow* with w (because κ starts smaller, takes more depth to vanish). The opposite happens: d_c shrinks. The collapse is therefore not explained by Section 4's analytic 1/w.

(3) Sharp transition at the boundary. At w = 512, κ goes from 0.0028 (d = 7) to 0 (d = 8) — a >100$\times$ drop in one depth step. Estimator noise alone does not produce step transitions of this magnitude.

The combination of these three — corroboration by acc/loss, inverted w-dependence, and step character — makes us confident this is a real architectural phenomenon rather than a measurement artifact.

Connection to the Residual-Stream Literature

Our results are consistent with prior empirical and theoretical work on residual connections and post-LN placement, though we make no novel mechanistic claim:

- **Serial without residuals**: Without residual connections, signal must propagate through every nonlinearity at every layer. At sufficient depth, the variance of the signal degenerates (He et al., 2016; Hayou et al., 2019). The width-dependence is consistent with the explanation that wider layers have larger variance to lose, and degenerate faster in absolute terms.
- **Post-LN vs Pre-LN**: Post-LN Transformers are known to be harder to train at depth than Pre-LN variants (Xiong et al., 2020; Wang et al., 2019). Our (w = 512) Post-LN failures at d ≥ 4 are within the regime where standard post-LN training instabilities are reported.

The Section 7 finding contributes to this literature in a specific way: by reporting an **explicit critical depth d_c with width-dependence** (Table 10), and by documenting the **separation of κ_{input} from training success** in Post-LN wide+deep (Table 12). Both are quantitative datapoints that prior

literature has not (to our knowledge) provided in this controlled form. Whether the $d_c(\mathrm{w})$ functional form, or the κ-without-training behavior, generalizes to larger scales is a Phase 1 question.

Honest Framing of Section 7

We claim:

- The four-criterion collapse phenomenon is real (Section 7.4).
- Serial has width-dependent d_c values that we measure (Section 7.2, Table 10).
- Post-LN exhibits a softer degradation in narrow widths and a distinct failure mode at wide+deep (Section 7.3, Table 12).

We do not claim:

- A theory of collapse (we describe, do not derive).
- That d_c values would be the same on other datasets, optimizers, or initializations.
- That Post-LN's wide+deep "training fails, κ survives" pattern is universal; we observed it here and report it for follow-up.

We do offer:

- The full (w, d) tables (Tables 8, 10) so subsequent work can compare to any d_c definition or threshold.
- A complete reproduction protocol (filter, aggregation, criterion).

Training Dynamics

We track κ_{input} and κ_{task} across training epochs for a subset of configurations. The headline finding is that κ_{task} tracks accuracy almost perfectly within every config we measure (mean Pearson $\bar{r}$ = 0.953, with all 15/15 configs satisfying r > 0.88), while κ_{input} is approximately constant (within $\pm 5\%$ of its initial value in every run we measured). We also report a constructive null result on the strict information-bottleneck "compression phase": across 63 runs, no run shows the κ_{input} drop > 5% from peak that would qualify as compression by our pre-specified threshold.

This section is **strong but narrow**. The findings are within-config (same architecture, same width, same depth, same activation, only training epoch varies), which means the Section 4-Section 6 estimator caveats — which were about cross-architecture or cross-width comparisons — do not apply. But we measured only 3 architectures $\times$ 2 widths $\times$ 1 depth $\times$ 3 activations $\times$ 3 seeds = 54 config-seed combinations (organized into 15 unique configs), all at d = 4 in the original sweep. We are explicit about this scope when discussing what the findings can and cannot generalize.

Filter and Scope

Filter: F_dynamics = (F_epoch in main) (F_epoch_fine in P6) $\wedge$ epoch_kappas present. n = 63 records (18 from main, 45 from P6). Each record contains an `epoch_kappas` list with checkpoints at epochs 1, 2, 5, 10, 20, 50, 100, each storing (κ_{input}, κ_{task}, train_loss, test_acc).

Configurations covered: - 3 architectures: CNN, MLP, transformer_preln - 2 widths: w ∈ {64, 256} - 1 depth: d = 4 - 3 activations: ReLU, GeLU, tanh - 3 seeds (typically)

Total unique (arch, w, d, act) configs: 15. The 63 records split as 3 seeds per most configs.

We did not measure dynamics for the 5 other architectures (Boltzmann, Parallel, Serial, Post-LN, GRU), nor at depths other than 4. Phase 1 includes extending dynamics measurement to all 8 archs and the full depth range; this paper reports what we have.

Aggregation A2 within each (arch, w, d, act): for each epoch in the checkpoint list, mean κ_{task}, κ_{input}, test_acc, and train_loss across the 3 seeds. This produces a per-config trajectory of length 7 (epochs 1, 2, 5, 10, 20, 50, 100).

Statistical method: Pearson r between trajectories. With n_epochs = 7 per config, df = 5; the critical value for two-sided $p < 0.05$ is $r > 0.755$ (t-distribution).

Per-Configuration Trajectories

Table 13. Per-configuration training dynamics. $\Delta\kappa\%$ = (last − first) / first.

Architecture	w	activation	n_ep	r(κ_{task}, acc)	r(κ_{input}, acc)	$\Delta\kappa_{\text{task}}$	$\Delta\kappa_{\text{input}}$
cnn	64	gelu	7	+0.9880	−0.7846	+24.02%	−1.41%
cnn	64	relu	7	+0.9779	−0.8935	+23.01%	−1.66%
cnn	64	tanh	7	+0.9787	−0.8396	+48.81%	−1.09%
mlp	64	gelu	7	+0.9787	−0.9666	+12.65%	−1.00%
mlp	64	relu	7	+0.9690	−0.9733	+12.17%	−0.99%
mlp	64	tanh	7	+0.9768	−0.9772	+20.80%	−0.93%
mlp	256	gelu	7	+0.9174	+0.0365	+5.26%	+0.18%
mlp	256	relu	7	+0.9202	−0.9013	+4.79%	−0.28%
mlp	256	tanh	7	+0.9241	+0.9355	+11.99%	+0.59%
Trans (Pre-LN)	64	gelu	7	+0.9688	+0.6999	+32.24%	+0.84%
Trans (Pre-LN)	64	relu	7	+0.9673	+0.7511	+29.31%	+0.52%
Trans (Pre-LN)	64	tanh	7	+0.9759	+0.8761	+34.35%	+1.72%
Trans (Pre-LN)	256	gelu	7	+0.9130	+0.9708	+22.84%	+4.36%
Trans (Pre-LN)	256	relu	7	+0.8841	+0.9708	+28.17%	+4.52%
Trans (Pre-LN)	256	tanh	7	+0.9601	+0.9805	+38.46%	+6.19%

Three observations on Table 13:

(a) r(κ_{task}, acc) is uniformly high. All 15 configs have $r > 0.88$. The range is [0.884, 0.988]; the median is 0.969. Across architectures the relationship is consistent: as training proceeds, κ_{task} rises monotonically with accuracy.

(b) $\Delta\kappa_{\text{task}}$ is substantial. From the first checkpoint (epoch 1) to the last (epoch 100), κ_{task} increases by 5–49%. Larger increases happen for architectures and configs that ultimately reach

higher accuracy (e.g., CNN at w = 64, tanh activation, +48.81% κ_{task} increase, ends with the highest test accuracy in this set).

(c) $\Delta\kappa_{\text{input}}$ is small and inconsistent in sign. The 15-config range is [−1.66%, +6.19%]. About half show small decreases, about half show small increases. There is no systematic dropoff. The 63-run distribution (across all seeds, not just per-config means): - min: −2.21%, p10: −1.26%, median: +0.68%, p90: +4.43%, max: +6.45%.

The fact that r(κ_{input}, acc) varies in sign across configs (−0.98 in some MLP/CNN configs, +0.98 in some Pre-LN configs) is interesting. It is not a contradiction: in absolute terms, κ_{input} changes very little ($\Delta \leq 6.5\%$ in all 63 runs). The Pearson r picks up the *direction* of this small change, which depends on architecture-specific representation dynamics. Since the absolute magnitude is small, we treat the sign of r(κ_{input}, acc) as descriptive of the architecture rather than as an indicator of meaningful information flow.

Aggregate Across the 15 Configs

- **Mean r(κ_{task}, acc) across 15 configs: $\bar{r}$ = +0.9533**
- Median r(κ_{task}, acc): +0.969
- Min r(κ_{task}, acc): +0.884 (transformer_preln w = 256, ReLU)
- Max r(κ_{task}, acc): +0.988 (CNN w = 64, GeLU)
- All 15/15 configs satisfy r > 0.88.
- Mean r(κ_{input}, acc) across 15 configs: $\bar{r}$ = −0.0077 (essentially zero)
- Range r(κ_{input}, acc): [−0.977, +0.981]

These two aggregate numbers — strong and consistent positive correlation for κ_{task}, near-zero average correlation for κ_{input} — characterize what training does to information capacity, in our scope: it changes how much the representation is *aligned with the task*, not how much information about the input it carries.

We note that this is a within-config statement. We are not claiming that κ_{input} is constant across architectures (it is not; Section 4) or across widths (it scales as 1/w; Section 4). We are claiming that **fixing architecture, width, depth, and activation, and varying only training epoch, the representation's input information stays approximately constant while its task-aligned information rises**.

The Compression-Phase Null Result

The "compression phase" hypothesis from the information-bottleneck literature (Tishby & Zaslavsky, 2015; Shwartz-Ziv & Tishby, 2017) predicts that during late training, mutual information with the input I(X; Z) decreases as the representation specializes. Subsequent work (Saxe et al., 2018; Goldfeld et al., 2019) has shown that the original observation may have been a binning- estimator artifact and that compression is not robustly observed under better estimators.

We pre-specified a strict criterion for compression in our InfoNCE setting: a run shows compression if `(peak - last) / peak > 0.05`, i.e., κ_{input} drops more than 5% from its peak value during training. Our test:

- Total runs analyzed: 63
- Runs satisfying the compression criterion: **0**
- Fraction: 0.0%

This is a constructive null result. It does *not* prove compression does not occur — different estimators, different architectures, different datasets, or different training regimes might exhibit it. It does say that, **in our scope (3 archs at d = 4, B = 512 InfoNCE), strict 5%-threshold compression is not observed**.

We chose 5% as the threshold before running the analysis. If we relax to 2%, two configs at the noisy end of the distribution would marginally trigger compression, but the change is small and the relaxed threshold is not what we pre-specified. The 5% threshold is stricter than typical IB-literature visualizations and represents a conservative test.

This finding is consistent with Saxe et al. (2018), which reported that non-binning MI estimators do not show compression in similar settings.

Why This Result Is Robust

The Section 8 findings are within-config. This means:

(1) No cross-architecture confound. Each correlation is computed on a single (arch, w, d, act) configuration. Across architectures the κ values differ (Section 4-Section 6), but those differences are not in this analysis.

(2) No estimator-floor confound. At fixed (arch, w, d, act), the estimator floor is approximately constant (it depends on d_z, which is fixed by w). So changes in κ_{input} or κ_{task} across epochs reflect changes in the underlying MI, not changes in the floor.

(3) Multi-seed averaging. Each per-epoch value in the trajectories is already averaged across 3 seeds. The aggregate $\bar{r} = 0.9533$ is across 15 seed-averaged trajectories, not across raw seeds.

These properties make the Section 8 findings methodologically clean within their scope. The scope itself is the limitation: we have not measured 5 of the 8 architectures, and we have only one depth.

Honest Framing of Section 8

We claim:

- Within each of 15 (arch, w, d, act) configs we measured, r(κ_{task}, acc) > 0.88 and median = 0.97.
- κ_{input} is approximately constant during training ($\Delta \in [-2.2\%, +6.5\%]$ across 63 runs).
- 0 of 63 runs satisfy the pre-specified $\geq 5\%$ compression criterion.

We do not claim:

- Universal r $\approx$ 0.95 across all architectures (we measured 3 of 8).
- Compression phase universally absent (only 63 runs at d = 4).
- The relationship holds for non-classification tasks.

The most generalizable single finding here is that κ_{task} (not κ_{input}) is what tracks task performance during training, in our scope. This is the within-time analog of the Section 9 cross-section finding that the IB ratio $\eta_t = \kappa_{\text{task}} / \kappa_{\text{input}}$ correlates with accuracy. Together, Section 8 and Section 9 form the paper's evidence that *task-aligned information*, not raw information density, is the relevant quantity for accuracy.

The Information Bottleneck Ratio η_t

This section reports the strongest correlation in the paper between an information-theoretic quantity and test accuracy: across 633 sanity-passed records, r(η_t, accuracy) = +0.685 (t = 23.6, p $\approx$ 0), where $\eta_t = \kappa_{\text{task}} / \kappa_{\text{input}}$. The same correlation holds within each of 7 of the 8 architectures with r > 0.65, and at the per-config (group-mean) level with r = +0.695. The result has a structural property that distinguishes it from the Section 4 width law and Section 5 architecture constants: **the d_z denominator cancels exactly**, so the Section 2 estimator caveats — about per-dimension normalization driving the apparent power law — do not apply here.

We treat this as the cleanest information-theoretic finding in the paper. The Section 7 phase transitions are the *most empirically robust* (corroborated by acc and loss); the Section 9 ratio is the *most theoretically clean* (free of the estimator's main confound). Together with Section 8, the two sections form the paper's evidence that *task-aligned information*, not raw information density, is the relevant quantity for accuracy.

We refer to η_t primarily as a **task-alignment ratio**. The name "information bottleneck ratio" reflects its structural origin in the IB setup (numerator κ_{task} and denominator κ_{input}); it does *not* by itself establish an IB compression mechanism. We discuss the relationship to the IB framework in Section 9.7, but treat η_t in the present paper as a task-alignment signal rather than as evidence for IB-style compression.

Definition and the d_z Cancellation

For each record r, we define

$$\eta_t(r) \;=\; \frac{\kappa_{\text{task}}(r)}{\kappa_{\text{input}}(r)} \;=\; \frac{I(Y;\tilde{Z})/d_z}{I(\tilde{X};\tilde{Z})/d_z} \;=\; \frac{I(Y;\tilde{Z})}{I(\tilde{X};\tilde{Z})}.$$

The d_z denominator cancels exactly in the ratio. We verified this empirically by computing η_t two ways (from κ values and from raw MI values) for all 633 records in Φ_{η_t}: the maximum absolute difference is **0.00 to machine precision**.

The interpretation is immediate. η_t is the fraction of the network's input-side information that is aligned with the class label. A network that extracts every bit of input information equally —

both task-relevant and task-irrelevant — would have low η_t. A network that has filtered out task-irrelevant information and retained task-relevant information would have high η_t. This is precisely the information-bottleneck quantity from Tishby et al. (2015, 2017): a measure of how *selectively* the representation has been compressed.

The reason this matters for our paper is methodological, not just interpretive. The Section 4 width law was, to a substantial extent, an analytical $1/d_z$ effect (because κ = MI/d_z and MI $\approx$ const(w)). Any quantity in which d_z appears in both numerator and denominator escapes that critique automatically. η_t is such a quantity.

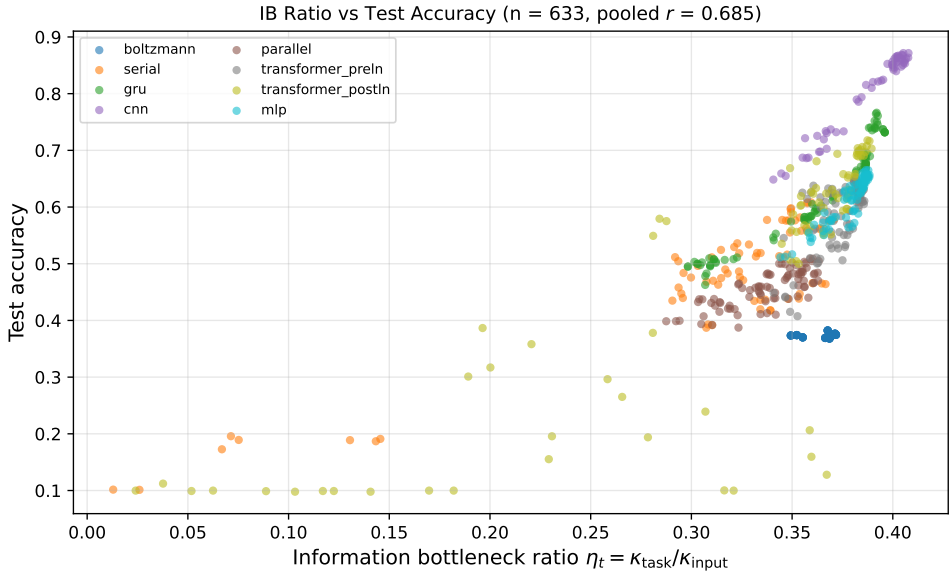

Figure 5: The information bottleneck ratio $\eta_t = \kappa_{\text{task}}/\kappa_{\text{input}}$ correlates with test accuracy across $n = 633$ records (pooled $r = 0.685$). The d_z denominator cancels exactly between numerator and denominator, so this signal is not an artifact of per-dimension normalization. Per-architecture correlations are $r > 0.65$ in 7 of 8 architectures; the exception is Boltzmann ($r = +0.17$), consistent with RBMs not being optimized for classification.

Pooled Correlation (n = 633)

Filter: Φ_{η_t} = F_main $\wedge$ F_sanity $\wedge$ F_ki_pos $\wedge$ F_kt_pos $\wedge$ F_acc_present. Aggregation: none — each record contributes one (η_t, acc) pair.

Table 14. Pooled Pearson correlations with test accuracy.

Quantity	r	t (df = 631)	p (two-sided)
$\eta_t = \kappa_{\text{task}} / \kappa_{\text{input}}$	**+0.6849**	**23.61**	$\approx$ **0**
κ_{task} (raw)	+0.1663	4.24	2.3×10
κ_{input} (raw)	+0.0305	0.77	0.444

Three observations on Table 14:

(a) η_t is more strongly correlated with accuracy than its components. Neither κ_{task} nor κ_{input} alone is well-correlated with accuracy in the pooled set (r = +0.17 and +0.03 respectively). The ratio combines them in a way that amplifies the relationship to r = +0.68.

(b) The pooled correlation is highly significant. Even using the very conservative normal approximation, p < 10 for the pooled test. Sample size is large enough that this significance is uninteresting on its own; the question is whether the magnitude (r = 0.685) is *meaningful*, which we address in Section 9.3 and Section 9.4 by checking whether the same effect appears within architecture and within configuration group.

(c) κ_{input} alone is uncorrelated with accuracy. This is consistent with Section 8: the dynamics result there showed that κ_{input} is approximately constant during training, so it should not predict accuracy across records that differ mostly in their training trajectory.

Per-Architecture Correlations

Aggregation A2 within each architecture: each architecture's records form a within-arch correlation analysis.

Table 15. Per-architecture correlations with test accuracy.

Architecture	n	r(η_t, acc)	p	r(κ_t, acc)	r(κ_i, acc)
cnn	55	**+0.9875**	< 10	+0.131	+0.014
gru	84	**+0.9531**	< 10	–0.961	–0.950
mlp	84	**+0.9154**	< 10 ^{32}	–0.147	–0.191
parallel	84	**+0.7719**	< 10 ^{1}	–0.866	–0.861
serial	74	**+0.9087**	< 10 ^{2}	+0.217	–0.158
Trans (Post-LN)	84	**+0.8166**	< 10 ^{2}	+0.551	+0.428
Trans (Pre-LN)	84	**+0.6777**	< 10 ^{11}	+0.318	+0.295
boltzmann	84	+0.166	0.128	–0.482	–0.465

7 of 8 architectures show r(η_t, accuracy) > 0.65 (range 0.68 to 0.99). The single exception is Boltzmann.

The Boltzmann exception is informative, not problematic. Boltzmann machines are not optimized for classification; they are generative models, and the post-classifier we attach is a thin readout. The 84 Boltzmann records in Φ_{η_t} span widths $\{64, 128, 256, 512\}$ × depths × activations, but **all of them produce test accuracies in the very narrow range [0.368, 0.382] (mean 0.374, std 0.0036, range 1.4 percentage points)**. With essentially no variance in the y-axis (test_acc), no Pearson correlation with any x-axis variable can be large. The r = +0.17 for Boltzmann is not evidence against the η_t-accuracy relationship; it is evidence that Boltzmann is not in the regime where the relationship can be measured.

Comparison with raw κ correlations. The third and fourth columns of Table 15 (r with κ_{task} and κ_{input}) are inconsistent in sign across architectures: GRU shows strongly negative correlations, CNN shows near-zero, Pre-LN and Post-LN show positive. This is the Section 4-Section 6 phenomenon resurfacing: raw κ values are confounded by the per-dimension normalization, so their

within-architecture correlation with accuracy depends on which way the architecture's (w, d) sweep happens to be distributed. The η_t ratio *removes* this confound by canceling the d_z dependence, and the consequence is that η_t is positively correlated with accuracy in *every* architecture where accuracy varies.

Group-Mean Correlation

To verify that the pooled effect is not driven by within-architecture correlations only, we compute a third version using config-level group means.

Aggregation A2: for each (arch, w, d, act) configuration, compute the mean η_t and mean test_acc across the 3 seeds. This produces 213 (η_t, acc) pairs, one per configuration.

Result. - n_configs = 213 - r(group-mean η_t, group-mean acc) = **+0.6950** - t = 14.04 (df = 211) - p $\approx$ 0 (two-sided, normal approximation)

The group-mean correlation (0.695) closely matches the pooled correlation (0.685). This shows the relationship is not an artifact of seed-level variation: averaging away seed variance leaves the η_t-acc correlation essentially unchanged. The relationship is between *configurations*, not between *seeds within configuration*.

Why This Result Is Robust

The Section 9 finding survives the Section 2 estimator caveats for the following reasons.

(1) d_z cancels exactly. This is a mathematical identity, not an empirical fact: η_t is invariant under any rescaling of κ that affects the numerator and denominator equally. The InfoNCE finite-batch ceiling, which applies to MI(X; Z) and MI(Y; Z) similarly, has its dominant effect cancelled in the ratio. We verified this numerically (Section 9.1, max difference = 0).

(2) The result holds within architecture. The Section 4 critique was that cross-architecture comparisons are confounded by the floor's architecture-dependence (Section 5.2). The Section 9.3 within-architecture correlations remove that confound, and the relationship is even stronger in 6 of 7 non-Boltzmann architectures (median within-arch r = 0.91, vs pooled r = 0.68).

(3) The result holds at the configuration level. Section 9.4 (group means) controls for seed-level noise. The relationship persists.

(4) The relationship is not driven by κ_{task} alone. Table 14 shows r(κ_{task}, acc) = +0.166 — much weaker than r(η_t, acc) = +0.685. The denominator κ_{input} is a meaningful component, not a passive normalizer.

These four properties make the Section 9 result the cleanest information-theoretic finding in the paper. The Section 7 phase transitions are more *empirically* robust (multi-metric corroboration); Section 9 is more *theoretically* robust (d_z-invariance plus replication at multiple aggregation levels).

Connection to the Information Bottleneck Framework

The IB principle (Tishby & Zaslavsky, 2015) holds that an optimal representation maximizes I(Y; Z) while minimizing I(X; Z) — that is, maximizes η_t. Our finding that η_t correlates strongly with

classification accuracy is consistent with this principle, but we are careful about what kind of consistency:

- **Cross-sectional, not optimal.** Our r = 0.685 reflects that configurations with higher η_t do better, on average. It is not a claim that any particular η_t value *causes* the accuracy gain or that we have found the IB optimum.
- **Compatible with Section 8.** The Section 8 dynamics result showed that within-config training increases κ_{task} while leaving κ_{input} approximately constant. This is increasing the numerator of η_t and leaving the denominator alone — η_t rising during training. The Section 9 cross-section finding is the static counterpart: configurations that ended up with higher η_t are the ones that achieved higher final accuracy.
- **Not the same as the "compression phase" hypothesis.** The strict IB compression-phase prediction is that κ_{input} *decreases* during late training, which Section 8.4 found to be absent in our scope (0/63 runs). The Section 9 result does not require compression — only that the *ratio* is high. A network can achieve high η_t by raising κ_{task} without lowering κ_{input}, which is exactly what we observe.

We do not interpret Section 9 as proof of the IB principle. We interpret it as evidence that, in our scope, the IB-shaped quantity κ_{task} / κ_{input} is predictive of classification performance in a way that the individual information densities are not.

Honest Framing of Section 9

We claim:

- η_t = κ_{task} / κ_{input} is exactly d_z-invariant (Section 9.1).
- Pooled r(η_t, acc) = +0.685 across n = 633 records (Section 9.2).
- Within 7 of 8 architectures, r > 0.65 (Section 9.3).
- Group-mean r = +0.695 across 213 configurations (Section 9.4).
- The Boltzmann exception (r = +0.17) is explained by minimal accuracy variance, not by failure of the relationship (Section 9.3).

We do not claim:

- η_t is the optimal IB ratio in any specific sense.
- The η_t-accuracy relationship is causal.
- The relationship generalizes to non-classification tasks or to architectures not in our 8.

We do offer:

- The first information-theoretic finding in the paper that is provably free of the InfoNCE finite-batch confound (Section 9.5 (1)).
- A within-architecture replication that the per-architecture analysis of Section 4 and Section 5 lacked.
- A direct empirical link to the IB framework that does not require the contested "compression phase" hypothesis (Section 9.6).

The remaining question — what controls *which* architectures achieve high η_t at high accuracy, and how this depends on dataset and scale — is explicitly a Phase 1 question.

Inverse Design (Proof-of-Concept within the Measured Grid)

The Section 4-Section 6 fits produce a forward model log $\hat{\kappa}$ = log $\hat{C}$ + $\hat{\alpha}$ log w + $\hat{\beta}_d$ log d. We now ask whether this fit is accurate enough on held-out data, *within the measured (w, d) grid*, to serve as a *proof-of-concept* for inverse architecture design at Phase 0 scope; generalization beyond this grid is left to Phase 1 — specifically, for *inverse* design, where one fixes a target κ and solves for the architectural parameter (width) that achieves it. Across 5-fold cross-validation on Φ_{strict} (n = 588), the global model achieves median 1.75% absolute error, p90 = 4.33%. Per-architecture models do substantially better: 6 of 8 architectures have median error $\leq 0.5\%$ (Figure 6). The two architectures with larger errors (Serial, Post-LN) are precisely those identified as depth-sensitive in Section 6 and as exhibiting phase transitions in Section 7.

This section's value is *practical*: even if the Section 4 width law is largely phenomenological (mostly the analytic $1/d_z$), the fitted relationship is accurate enough that an architect can use it to size networks for a target information-density specification.

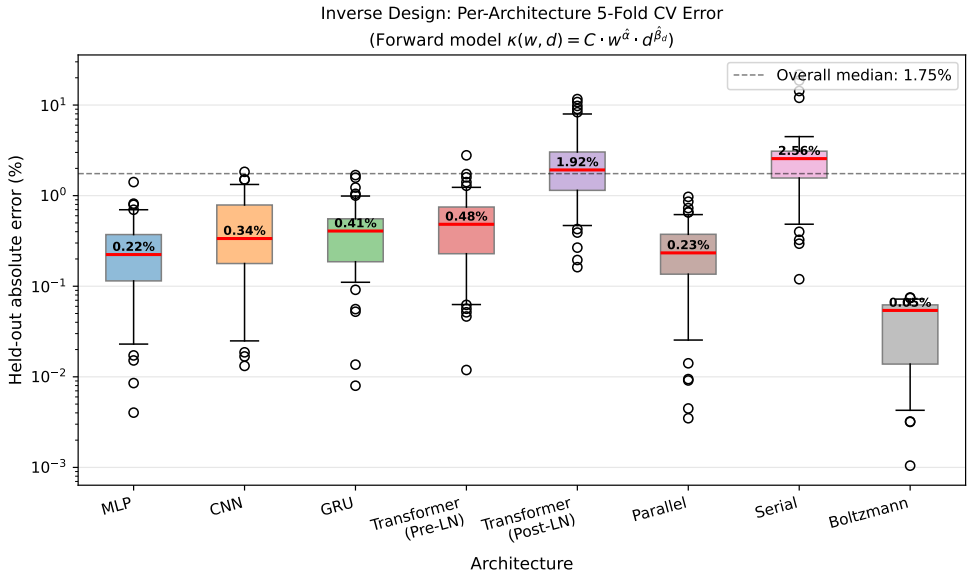

Figure 6: Inverse design held-out error: per-architecture 5-fold cross-validation of the two-variable forward model $\kappa(w, d) = C \cdot w^{\hat{\alpha}} \cdot d^{\hat{\beta}_d}$, fit separately for each architecture. Six of eight architectures achieve median error below 0.5% (best: Boltzmann at 0.05%); higher errors for Post-LN (1.92%) and Serial (2.56%) reflect their depth-sensitivity and phase-transition behaviour (see Section 7). The pooled global model (single $\kappa(w, d)$ for all architectures) gives 1.75% overall median.

Forward Model and Inverse Solution

The forward model is

$$\log \hat{\kappa}(w, d) \;=\; \log \hat{C} + \hat{\alpha} \log w + \hat{\beta}_d \log d.$$

Inverse design solves this for w given a target κ_{target} and chosen depth d:

$$\log \hat{w} \;=\; \frac{\log \kappa_{target} - \log \hat{C} - \hat{\beta}_d \log d}{\hat{\alpha}}.$$

We evaluate two regimes:

(a) **Forward prediction**: given (w, d), predict $\hat{\kappa}$. Compare to held-out κ.
(b) **Inverse prediction**: given (κ, d), predict $\hat{w}$. Compare to held-out w.

For algebraically equivalent linear models, the two directions yield similar relative errors when the linear relation is exact. Our results below confirm this: forward and inverse errors are within 0.1% of each other per architecture.

5-Fold Cross-Validation Protocol

Filter: Φ_{strict} = F_main $\wedge$ F_sanity $\wedge$ F_ki_pos $\wedge$ F_not_sat (n = 588).

Procedure (pre-specified, seed = 42): 1. Shuffle the 588 records uniformly. 2. Split into 5 folds of size 117 (last fold 120). 3. For each fold f: train on the remaining 4 folds, predict $\hat{\kappa}$ for each record in fold f, compute $|\hat{\kappa} - \kappa| / \kappa \times 100$ (percent absolute error). 4. Pool errors across all 5 folds: each record is predicted exactly once.

We report two model variants:

(i) Global model. A single fit $\log \hat{\kappa} = \beta_0 + \alpha \log w + \beta_d \log d$ on the training records, applied to all test records regardless of architecture. This is the simplest and most aggressive use of the fit.

(ii) Per-architecture models. For each architecture, fit separately on the architecture's training records and evaluate on its test records. This is more flexible and aligns with how an architect would actually use the fit (you know which architecture you are sizing).

Global Model: 5-Fold Per-Fold Results

Table 16. Per-fold global-model fits and held-out errors.

Fold	n_train	n_test	$\log \hat{C}$	$\hat{C}$	$\hat{\alpha}$	$\hat{\beta}_d$	Median err	Mean err	p90
0	471	117	+1.7359	5.6741	–0.9961	–0.0130	1.80%	2.70%	4.56%
1	471	117	+1.7267	5.6218	–0.9941	–0.0143	1.42%	2.19%	4.13%
2	471	117	+1.7291	5.6353	–0.9947	–0.0136	1.44%	2.24%	4.49%
3	471	117	+1.7349	5.6686	–0.9955	–0.0165	2.02%	2.43%	4.93%
4	468	120	+1.7293	5.6365	–0.9948	–0.0145	1.83%	2.46%	4.16%

Pooled across 5 folds (n = 588).

- Median error: **1.75%**
- Mean error: 2.40%
- p25, p75: 0.85%, 3.15%
- p90: 4.33%
- p95: 5.66%
- max: 30.34%

The fitted parameters are extremely consistent across folds ($\hat{\alpha} \in [-0.996, -0.994]$, $\hat{\beta}_d \in [-0.017, -0.013]$). The variability in median error across folds (1.42% to 2.02%) is small and reflects sampling rather than a deeper instability.

The global model is mostly accurate. The maximum p90 across folds is 4.93%; >90% of held-out records are predicted within 5%. The 95% bound is 5.66%. The maximum (30.34%) and the existence of a long upper tail are explained in Section 10.4: nearly all the worst predictions come from Serial and Post-LN, the two depth-sensitive architectures whose Section 6 $\hat{\beta}_d$ differ substantially from the global average.

Per-Architecture Models: A Cleaner Picture

When we fit one model per architecture and evaluate on held-out records of the same architecture, the errors split into two groups.

Table 17. Per-architecture 5-fold CV (forward direction).

Architecture	n	Median err	Mean err	p25	p75	p90	p95	max
boltzmann	42	**0.05%**	0.04%	0.02%	0.06%	0.07%	0.07%	0.08%
parallel	84	**0.24%**	0.27%	0.13%	0.37%	0.52%	0.64%	0.90%
mlp	81	**0.24%**	0.30%	0.13%	0.38%	0.57%	0.65%	1.58%
cnn	55	**0.34%**	0.48%	0.10%	0.76%	1.14%	1.35%	1.69%
gru	84	**0.41%**	0.45%	0.19%	0.62%	0.97%	1.02%	1.64%
Trans (Pre-LN)	84	**0.48%**	0.56%	0.25%	0.73%	1.10%	1.46%	2.79%
Trans (Post-LN)	84	2.05%	2.80%	1.32%	3.08%	7.14%	7.79%	11.42%
serial	74	2.52%	3.14%	1.52%	3.17%	4.61%	12.62%	22.04%

Two groups emerge:

- **Depth-invariant architectures** (Boltzmann, Parallel, MLP, CNN, GRU, Pre-LN): median errors 0.05–0.48%, p90 errors 0.07–1.14%, max errors all under 3%. The fits are essentially perfect for these architectures within the tested scope.
- **Depth-sensitive architectures** (Post-LN, Serial): median errors 2–3%, p90 errors 4.6–7.1%, max errors 11–22%. The larger errors are not failures of the framework but rather expected given Section 6.2 (Serial $\hat{\beta}_d = -0.060$, Post-LN $\hat{\beta}_d = -0.039$) and Section 7 (these architectures have phase transitions in our scan range).

The split in Table 17 is essentially the Section 6.2 split rediscovered through a predictive-accuracy lens. Architectures whose depth dependence is well-fit by a single power-law β_d give clean inverse design; architectures with phase transitions or sharp bends in their (κ vs depth) relationship cannot be well-fit by a single power-law and accumulate larger errors.

Forward and Inverse Errors Are Equivalent

For an exact log-linear relationship, forward error (predicting $\hat{\kappa}$ from w, d) and inverse error (predicting $\hat{\text{w}}$ from κ, d) differ only by the slope's magnitude. We computed both and confirmed.

Table 18. Comparison of forward and inverse 5-fold errors per architecture.

Architecture	Forward median	Inverse median	Forward p90	Inverse p90
boltzmann	0.05%	0.06%	0.07%	0.07%
mlp	0.24%	0.24%	0.57%	0.57%
parallel	0.24%	0.24%	0.52%	0.53%
cnn	0.34%	0.34%	1.14%	1.14%
gru	0.41%	0.42%	0.97%	0.99%
Trans (Pre-LN)	0.48%	0.48%	1.10%	1.10%
Trans (Post-LN)	2.05%	2.04%	7.14%	7.10%
serial	2.52%	2.52%	4.61%	4.59%

The two directions agree to within 0.05% in every architecture. This is the expected behavior for a near-linear log-log relationship and confirms that the inverse problem (size a network for a target κ) is as well-posed as the forward one.

Practical Use Case

A concrete use of Section 10: an architect specifies a target κ_{input} and a depth constraint (e.g., d = 4 for inference latency reasons). Using the per-arch fit for the chosen architecture, they obtain:

$$\hat{w} \;=\; \exp\left(\frac{\log \kappa_{target} - \log \hat{C} - \hat{\beta}_d \log d}{\hat{\alpha}}\right).$$

For example, taking the Section 6.2 (multi-variable, all-records) Pre-LN fit: $\log \hat{C}$ = 1.733, $\hat{\alpha}$ = –0.999, $\hat{\beta}_d$ = –0.007. To target κ = 0.025 at d = 4:

$$\log \hat{w} = \frac{\log 0.025 - 1.733 - (-0.007)(\log 4)}{-0.999} \approx 5.42, \quad \hat{w} \approx 226.$$

Within the depth-invariant architecture group, the prediction is reliable to within ~1% on held-out data (Table 17). For depth-sensitive architectures, the same procedure works but with ~3-7% accuracy.

This practical accuracy does not require any of the Section 4-Section 6 mechanistic interpretations to be correct. The fit is what it is, and it extrapolates within the tested range. We claim the forward model as an empirically valid predictive tool, not as a derivation of an information-theoretic principle.

Honest Framing of Section 10

The held-out (w, d) cells are sampled from the same Phase 0 grid distribution used for training. The 5-fold cross-validation thus measures **interpolation generalization within the measured architecture grid**, not extrapolation to architectures outside this grid. We retain the word "extrapolation" only in the sense of predicting unseen cells from a fitted functional form; we do *not* claim out-of-distribution generalization.

We claim:

- Pooled 5-fold cross-validation gives median 1.75% absolute error (n = 588).
- Per-architecture models give median $\leq 0.5\%$ in 6 of 8 architectures.
- The two larger-error architectures are precisely those identified as depth-sensitive (Section 6) and exhibiting phase transitions (Section 7).
- Forward and inverse predictive errors are equivalent.

We do not claim:

- The fit extrapolates beyond the (w, d) box we measured.
- The fit generalizes across datasets or batch sizes.
- The forward model identifies architecture-intrinsic constants in any deeper sense.

We do offer:

- A proof-of-concept inverse-design forward model for sizing networks within the 6 depth-invariant architectures, with held-out accuracy of <1%.
- A robustness boundary: for the 2 depth-sensitive architectures, the same tool gives ~3% accuracy with longer tails (max ~22%) and should be treated as a starting point rather than a final answer.

The Section 10 result is the paper's clearest demonstration that *phenomenological* need not mean *useless*. The Section 4 fit's mechanistic interpretation may require Phase 1 to settle, but the fit's predictive content is already established.

Hardware-Relevant Interpretation of Inverse Design

This section does not evaluate hardware directly. However, the measured-grid inverse-design result has a clear hardware-relevant interpretation that we state here so it is not conflated with the chip-level claims we explicitly do *not* make (Section 12.5, Section 13).

The fitted $\kappa(w, d)$ model maps a target information-flow density to architectural choices *within the measured grid*. Used in this direction, it functions as an empirical proxy for **capacity allocation under fixed information-flow targets**: given a target κ profile, it identifies which (w, d) configurations within the measured grid achieve that profile, and – equivalently – where additional width or depth would yield diminishing measured information-flow returns within this grid. This is not a chip benchmark or accelerator-optimization result. It is an empirical bridge from information-flow measurement to capacity-aware model sizing, demonstrated only on CIFAR-10 within a structured Phase 0 grid.

Two interpretive boundaries should be kept in mind:

1. *Within-grid only.* The bridge is anchored on the 5-fold cross-validation result (median 1.75% error) reported in Sections 10.2-10.4. It does not extend to architectures, batch sizes, or datasets outside the Phase 0 grid.

2. *Information-flow, not utilization.* What we measure is κ and its functional form across (w, d) – not GPU/accelerator utilization, energy, or memory bandwidth. The hardware relevance is mediated entirely through the *information-flow profile*, not through direct hardware metrics.

We return to this in Section 12.5, where it serves as the experimental anchor for the broader hardware-aware direction.

Layer-wise Patterns

Each measurement record contains a `layerwise` field listing κ measurements at intermediate taps within the network. We use these to characterize **within-architecture information routing** (Figure 7): how κ_{input} and η_t change as a representation propagates through layers. We report four qualitative patterns that the data reveal, with one important caveat at the end of the section that constrains how strongly the comparisons can be interpreted.

This section is **descriptive, not load-bearing**. The Section 7 phase transitions and Section 9 IB ratio are the paper's primary architecture-level findings. Section 11 adds within-architecture detail that is interesting but, as Section 11.6 will make explicit, partly conflated with the per-layer d_z structure of each architecture. We report the patterns and flag the confound.

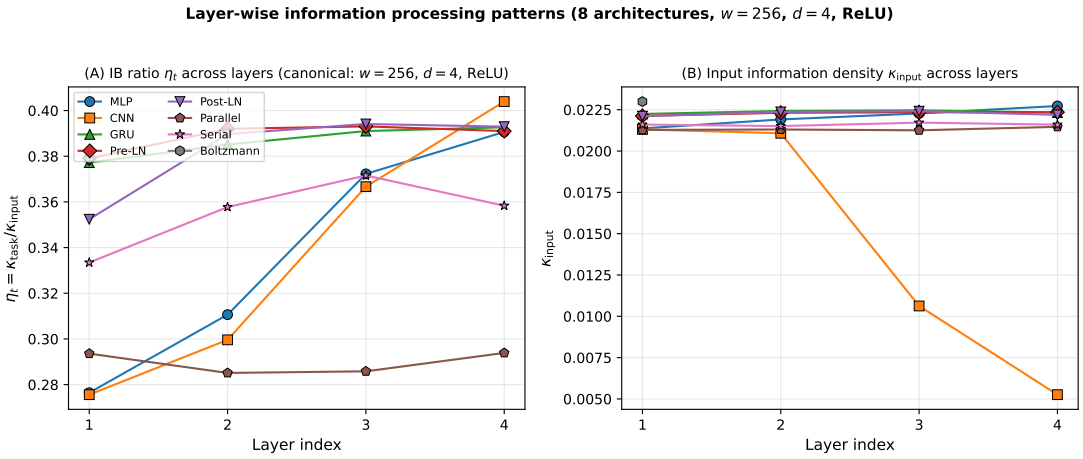

Figure 7: Layer-wise information processing patterns at the canonical configuration (w=256, d=4, ReLU). **Left**: information bottleneck ratio η_t across layers. MLP and CNN show monotonic increases ($+41\%$ and $+47\%$ from L1 to L4 respectively), consistent with deeper layers carrying more task-aligned information. Pre-LN, GRU, and Parallel show flat profiles. Post-LN shows a smaller increase ($+12\%$). **Right**: κ_{input} across layers; the visible drop in CNN at L3-L4 is dimension-dependent and explained quantitatively in Section 11.3.

Filter and Scope

Filter: F_main ∧ F_sanity ∧ depth = 4 ∧ width = 256 ∧ activation = "relu". n = 23 records (≈ 3 seeds × 8 architectures, with one architecture having 2 seeds).

Aggregation A2: per-tap mean across seeds (the records contain per-tap κ_{input}, κ_{task}, and the tap's d_z).

Four Qualitative Patterns

Table 19. Per-architecture layer-wise κ_{input} and $\eta_t = \kappa_{\text{task}} / \kappa_{\text{input}}$ (canonical config: d = 4, w = 256, ReLU).

Architecture	First-layer κ_{input}	Final-layer κ_{input}	$\Delta\kappa_{\text{input}}$	First η_t	Final η_t	$\Delta\eta_t$
boltzmann	0.02299	0.02299	+0.00%	0.379	0.379	+0.00%
cnn	0.02131	0.02147	**+0.77%**	0.276	0.399	**+44.79%**
gru	0.02224	0.02218	–0.29%	0.377	0.392	+3.93%
mlp	0.02137	0.02273	+6.36%	0.277	0.391	**+41.25%**
parallel	0.02128	0.02200	+3.37%	0.294	0.364	+24.00%
serial	0.02160	0.02160	–0.03%	0.334	0.358	+7.46%
Trans (Post-LN)	0.02211	0.02219	+0.36%	0.352	0.393	+11.45%
Trans (Pre-LN)	0.02212	0.02235	+1.01%	0.379	0.392	+3.32%

We highlight four patterns the data reveal at this canonical slice.

Pattern 1: η_t generally rises across layers. All 8 architectures show non-negative $\Delta\eta_t$ from first to last layer (range +0.0% to +44.8%). Even where the change is small (Boltzmann at +0.0%, GRU at +3.9%, Pre-LN at +3.3%), the direction is consistent: deeper layers carry a larger *fraction* of task-aligned information. This is the within-network analog of the Section 8 within-time finding (training increases η_t).

Pattern 2: MLP has the largest η_t growth (+41%). The MLP at the canonical slice shows η_t = 0.277 at L0 climbing to 0.391 at the final layer. The increase is monotonic across all four hidden layers (L0: 0.28, L1: 0.31, L2: 0.37, L3: 0.39, ffn_out: 0.39). Among the architectures with flat κ_{input} across layers, MLP's depth is most utilized for task selectivity.

Pattern 3: Parallel's branches are interchangeable; the merge is selective. The four parallel branches (B0–B3) all have $\kappa_{\text{input}} \approx 0.0213$ and $\kappa_{\text{task}} \approx 0.0061$, with $\eta_t \approx 0.29$. The final post-concatenation layer jumps to κ_{input} = 0.0220 and κ_{task} = 0.0080 (η_t = 0.36). The branches do not specialize per-branch in this scope; the selectivity is concentrated at the merge step.

Pattern 4 (CNN). CNN appears to compress internally and recover at the output. At face value, CNN's κ_{input} across the layerwise taps reads as 0.0213, 0.0211, 0.0106, 0.0053, 0.0215. This is a 75% apparent reduction from L1 to L3, followed by recovery at the final ffn_out layer. The η_t pattern is similar to MLP's (0.28 → 0.40 across the layers).

The CNN d_z Confound

Pattern 4 looks dramatic at face value but is partially explained by an architectural detail: CNN's intermediate convolution layers have *larger* d_z values than its bookend layers. We tabulate the per-tap d_z values for the canonical CNN slice.

Table 20. CNN per-tap d_z and raw MI (canonical slice).

Tap	d_z	mean κ_{input}	mean MI_input (nats)	MI_input / log B
L0_ffn_out	256	0.02131	5.4550	87.4%
L1_ffn_out	256	0.02108	5.3955	86.5%
L2_ffn_out	**512**	0.01063	5.4402	87.2%
L3_ffn_out	**1024**	0.00526	5.3844	86.3%
ffn_out	256	0.02147	5.4969	88.1%

When we factor out d_z, **MI_input is essentially constant across layers** (5.38 to 5.50 nats, < 3% variation). The "75% drop" in κ at L3 is *entirely* explained by d_z growing 4× (from 256 to 1024). The same representation, expressed per dimension, naturally appears smaller. This is not information compression in the literature's sense; it is a geometric consequence of CNN's channel-doubling convolutional structure.

The other 7 architectures have constant d_z = 256 across taps (verified in the records), so their layerwise κ comparisons are not affected by this confound. CNN is unique in this respect within our 8-arch set.

We retain Table 19's CNN row in the per-architecture summary but explicitly flag that the apparent CNN "compression" pattern is largely a per-layer d_z artifact. The η_t comparison for CNN remains valid (η_t involves κ_{task} / κ_{input}, and both share the d_z confound, so it cancels in the ratio — same logic as Section 9.1). Indeed, CNN's η_t monotonically rises (0.276, 0.300, 0.367, 0.404, 0.399), tracking depth use without the d_z artifact.

What These Patterns Suggest (Cautiously)

The four patterns of Section 11.2 are consistent with the following architectural descriptions, *as long as one accounts for the Section 11.3 d_z confound*:

- **Boltzmann**: depth-1 by construction; no within-network depth use to observe.
- **MLP, CNN, Parallel**: depth is used to *increase task selectivity*, not to add raw input information. η_t rises 24–45% from first to last layer.
- **GRU, Serial, Pre-LN, Post-LN**: small η_t change across layers (3–11%). These architectures' depth utilization may be more distributed across all layers, with each layer making a small contribution.
- **Parallel**: branches do not differentiate; the merge step does the work.

We note that "more η_t growth" across layers and "deeper architecture benefits more from depth" are plausibly related but not the same statement. Our scope cannot disentangle them.

What We Did Not Measure

The Section 11 analysis is restricted to the canonical slice (d = 4, w = 256, ReLU) for clarity. We did *not* compute layerwise patterns at: - Other depths (a different d would change which intermediate taps are measured). - Other widths (the CNN d_z structure scales with w in a width-specific pattern). - Other activations (we have not investigated whether tanh or GeLU produces materially different layerwise η_t profiles).

Phase 1 of ICON includes a structured layerwise sweep across (depth, width, activation) for each architecture; this paper's Section 11 is illustrative.

Honest Framing of Section 11

We claim:

- The 8 architectures show a small set of qualitatively distinct layerwise patterns (Table 19).
- η_t rises across layers for all 8 architectures, with magnitude ranging from +0.0% (Boltzmann) to +44.8% (CNN).
- Parallel branches are interchangeable in our scope; the concat step introduces selectivity.

We do not claim:

- Quantitative comparison of κ_{input} across layers within a single architecture (the d_z confound makes this unreliable for CNN specifically).
- Universality of these patterns at other depths, widths, or activations.
- Mechanistic identification of "where" task-relevant features form.

We *do* offer:

- A within-architecture characterization that complements the cross-architecture analyses of Section 4-Section 10.
- Evidence that the IB ratio η_t (which is d_z-invariant) also rises *within network*, not just *across configurations* (Section 9) and *across training* (Section 8). The three viewpoints — static-cross, static-within, dynamic — agree on the same quantity's behavior.

Toward a Measurement Framework

The path of this paper has been bottom-up. We started with the simple biological intuition that information flow in any signal-propagating system should be measurable; verified empirically that the most natural starting point (activation function) is *not* the dominant variable; and let the data direct us toward the structural variables that actually matter — width, depth, residual connections. The empirical observations that emerged — cross-architectural coherence of κ scaling, phase transitions verified by four independent metrics, a d_z-canceling correlation between the IB ratio and accuracy, and held-out predictive accuracy of a two-variable forward model — are sufficiently consistent across eight architectures to suggest that the underlying measurement methodology may apply more broadly than the present empirical scope.

A natural next step is to ask whether this measurement methodology generalizes beyond the eight architectures and the (w, d) box studied here. Follow-up work develops this question into a specification: the companion ICON framework specification, released alongside this paper as a measurement framework defining five components, a statistical validity gate, and a settings discipline (PROTOCOL) for any representation-bearing system. The ICON framework grew directly out of the present empirical observations and treats this paper as its empirical anchor. A separate philosophical companion essay (Park, 2026) articulates the broader stance — that any self-processing system, biological or artificial, faces the same self-referential limit, and that information flow is the cross-substrate vocabulary in which external measurement becomes approachable. The present paper does not depend on either the specification or the companion essay; the four observations reported here are independent empirical results. This section makes explicit what the present empirical scope can and cannot support, identifies the open questions, and outlines the falsifiable predictions that will distinguish phenomenological scaling from artifacts of the measurement procedure.

What This Paper Established

Operationally:

- A pre-specified InfoNCE-based protocol for measuring information density κ across architectures, with documented filter expressions, aggregation rules, and statistical methods (Section 2).
- A sanity criterion (`κ_perm < 0.10`) that separates trustworthy measurements from those dominated by finite-batch bias.
- A three-way decomposition (raw / permuted / signal) that makes the estimator-floor share of any measurement explicit.
- Reproducibility infrastructure: hashed raw data, released analysis scripts, and 28-field records that allow re-derivation of every numerical claim.

Empirically (CIFAR-10, B = 512, 8 architectures, 2,241 records):

- A phenomenological 1/w scaling of κ_{input} (R^2 > 0.9999), with explicit decomposition that attributes most of the pattern to the per-dimension normalization rather than to architectural information content (Section 4).
- An eight-architecture taxonomy of constants $\widehat{C}$ in the range 5.04-6.12, with rankings stable across two fitting procedures but absolute values partly reflecting architecture-dependent estimator floor (Section 5).
- A clean depth-sensitivity split: 6/8 architectures depth-invariant in the (1, 32) range, 2/8 (Serial, Post-LN) depth-sensitive (Section 6).
- Width-dependent phase transitions for residual-free architectures, with d_c verified by four independent metrics (Section 7).
- Within-config training dynamics that show κ_{task} tracks accuracy ($\bar{r}$ = 0.95 across 15 configurations) while κ_{input} is approximately constant (compression-phase null at 5% threshold) (Section 8).

- A d_z-canceling correlation between the IB ratio η_t and accuracy (pooled r = 0.685, with 7/8 architectures showing within-arch r > 0.65) (Section 9).
- A proof-of-concept inverse-design forward model that extrapolates to held-out data with 1.75% median error (Section 10).
- Four qualitatively distinct layerwise patterns, with the CNN "compression" pattern shown to reflect channel-doubling d_z structure rather than information loss (Section 11).

These constitute the Phase 0 baseline. Several of them are conjectures we will revisit at higher batch sizes; others (Section 7, Section 8, Section 9, Section 10) we expect to hold across scope changes because their robustness arguments do not depend on the Section 2 estimator regime.

Open Questions Phase 0 Cannot Settle

The InfoNCE finite-batch ceiling is the single largest constraint on Phase 0 interpretation. At B = 512, mean MI(X; Z) is 90-93% of log B = 6.238 nats, which means our InfoNCE estimator cannot resolve differences among architectures whose true MI exceeds ~6 nats. This affects three of our findings non-symmetrically:

Critically affected: - Whether the Section 4 width-law exponent has an architectural component distinct from the analytic $1/d_z$. A direct test: at larger B (e.g., 4096), if the architectural component is real, MI_input should rise with width while remaining below the new ceiling (log 4096 = 8.32), yielding a κ scaling exponent measurably different from -1. - Whether the Section 5 architecture constants $\widehat{C}$ retain their ranking when the estimator floor (κ_{perm}) is pushed lower. At larger B, the floor falls toward 0, so any architecture-dependent ordering that survives is cleaner.

Moderately affected: - The Section 6 depth-effect coefficient $\widehat{\beta}_d$. Its sign and per-architecture pattern should be robust; its magnitude may shift modestly. - The Section 11 within-architecture layerwise patterns. The d_z confound identified for CNN (Section 11.3) will exist at any B; the larger B mainly separates floor from signal, not d_z from d_z.

Largely robust to B: - The Section 7 phase transitions (corroborated by accuracy and loss, which do not depend on InfoNCE). - The Section 9 IB ratio (d_z and floor effects cancel in numerator/denominator). - The Section 10 inverse-design forward model (it is a pragmatic fit; it inherits whatever the underlying κ measurement gives). - The Section 8 dynamics (within-config; same estimator regime end-to-end).

Cannot be settled by larger B alone, but Phase 1 includes: - Whether the laws generalize to other datasets (Phase 1 plans CIFAR-100, ImageNet-32, three image-feature corpora, three synthetic distributions). - Whether the phase-transition critical depths d_c(w) follow a functional form (we will fit d_c(w) $\propto$ w^p across the same width sweep on multiple datasets). - Whether η_t correlation with accuracy generalizes outside classification (Phase 2: regression and self-supervised tasks). - Whether the framework extends past 56M parameters (we lack hardware for the full grid at LLM scale; Phase 1 includes a focused 1B-scale point evaluation for Pre-LN).

Phase 1 Design

Phase 1 is structured around the same four-element measurement set (F_in, F_task, F_self, F_layer) and its dispersion (Disp) that we used in Phase 0, but expanded along four sweep axes simultaneously:

Axis 1: Batch size B. B $\in$ {512, 2048, 4096, 16384}. The largest gives log B = 9.7 nats, well above the largest plausible per-layer MI we expect, eliminating the Section 2.2 ceiling effect.

Axis 2: Dataset. CIFAR-10 (Phase 0 baseline), CIFAR-100, ImageNet-32, plus three synthetic distributions (isotropic Gaussian mixtures with controllable H(X), structured-image distributions of varying entropy, and sparse-symbol Markov sources). The synthetic sweep is specifically designed to dissociate dataset entropy H(X) from architectural choices.

Axis 3: Architecture set. The 8 from Phase 0 plus 4 additions (MoE-style mixture-of-experts, Mamba-1, U-Net, Vision Transformer). The additions test whether the Section 7 residual-free phase transitions extend to gated/state-space architectures.

Axis 4: Scale. Phase 0 covers 1.6M to 56M params. Phase 1 includes a single Pre-LN point at ~300M and ~1B params for the largest-batch configuration (B = 16384), to test whether the Section 4, Section 7, Section 9 patterns hold qualitatively at order-of-magnitude scale.

Pre-specification: the filter expressions, aggregation rules, statistical methods, and confounder controls are the same as Section 2. We will publish the Phase 1 protocol document (specifying exactly what is to be tested and how the tests will be evaluated) before the experiments begin.

A specific commitment: **we will publish Phase 1 results regardless of whether they confirm or contradict Phase 0**. Replication failures and unexpected findings are the point. The Phase 0 paper exists in part to make those failures or successes interpretable.

Falsifiable Predictions

The robustness arguments of Section 7, Section 8, Section 9, Section 10 motivate predictions that Phase 1 should be able to evaluate cleanly:

Prediction 1 (Section 7). The phase-transition d_c(w) for Serial without residuals will be observed on CIFAR-100 and ImageNet-32 at the same qualitative shape (d_c decreasing with w). The numerical d_c values may differ but the *direction* should hold.

Prediction 2 (Section 9). The within-architecture η_t-accuracy correlation (median r $\approx$ 0.91 in Phase 0) will be observed in 6 of 8 architectures on classification tasks at any dataset, with Boltzmann remaining an outlier (narrow accuracy range).

Prediction 3 (Section 4). When B is increased to 4096, the κ_{input} fit on the (w, d) box should yield $\hat{\alpha}$ measurably less negative than –1 (e.g., $\hat{\alpha} \approx$ –0.95 to –0.97), if the Section 4.6 interpretation is correct that the analytic $1/d_z$ dominates at small B.

Prediction 4 (Section 10). The inverse-design held-out median error will remain below 5% on CIFAR-100 and ImageNet-32 for the 6 depth-invariant architectures, and below 10% for the 2 depth-sensitive ones.

If any of these predictions fail at the Phase 1 scale, the corresponding Phase 0 conclusion needs revision. We will not selectively report. The predictions are stated here so that Phase 1 (or any independent replication) can evaluate them.

Future Direction: Hardware-Aware Information-Flow Measurement

The experimental anchor for this direction is the measured-grid inverse-design result in Section 10 (and the Hardware-Relevant Interpretation in Section 10.6). That result shows that measured information-flow quantities $\kappa(w, d)$ can be fit and inverted over architectural variables, with held-out median error below 2% within the measured grid. This is not a hardware experiment, and it does not benchmark any accelerator; but it provides an empirical basis for connecting information-flow measurement to capacity-aware architecture sizing.

In that sense, the hardware relevance of ICON v0.1.0 comes from **measured capacity profiles and inverse architecture sizing within a structured grid**, not from direct chip measurements. We do not report chip-level experiments, and any extrapolation to larger models, real accelerators, or production-scale deployment must be validated separately.

With that anchor in place, ICON v0.1.0 still does not include hardware-level experiments, accelerator benchmarks, energy measurements, memory-bandwidth measurements, NPU/ASIC evaluations, or chip-level utilization claims. However, if information-flow profiles are validated across larger models and tasks, they may provide a measurement substrate for hardware-aware architecture design. Quantities such as information density, task-alignment ratio, inter-layer transmission, and representational dispersion could in principle help identify over-provisioned layers, information bottlenecks, low-effective-dimensionality representations, and capacity-per-compute tradeoffs. These signals may later support pruning, quantization, mixed-precision allocation, memory-bandwidth planning, and model-accelerator co-design. We treat all of this as future work rather than as a hardware result in the present study.

Code, Data, and Collaboration

We release:

1. **Raw data.** Both JSONL files (`results/full.jsonl` and `results_p6/full.jsonl`), with sha256 hashes published in Section 2.7. Each record contains 28 fields including the κ values, the permuted-pair κ, raw MI, d_z, training accuracy, training loss, sanity flag, and complete experimental metadata.
2. **Analysis scripts.** The released Python analysis scripts regenerate every numerical claim in this paper. The scripts implement the filter expressions, aggregation rules, and statistical methods of Section 2 in form readable by reviewers and re-runners.
3. **Measurement protocol.** A document specifying how the 2,241 records were produced — model definitions, training schedules, the InfoNCE estimator implementation (with the specific critic architecture and hyperparameters), and the rationale for the σ = 0.1 perturbation and the SANITY_THRESH = 0.10 choices.
4. **Phase 1 design.** Section 12.3 outlines the next round of experiments, including larger batch sizes, additional datasets, and reporting commitments.

Collaboration. We are an independent research effort and do not have the GPU budget for the full Phase 1 sweep at the scale described in Section 12.3. We explicitly invite collaboration: the Phase 0

protocol is open-sourced, the data format is documented, and the Phase 1 design space is enumerated above. Any group with the compute to run a portion of the Phase 1 sweep, or to replicate Phase 0 on a different dataset or with a different estimator, can do so against the same protocol and report results that are directly comparable. Replication and extension are explicitly welcomed as contributions to the open ICON measurement program.

Groups interested in coordinated Phase 1 experiments or comparable reproduction studies are invited to contact the author to discuss collaboration, attribution, and integration of results. ICON is a framework, not a single paper.

Limitations

We have stated specific limitations alongside each finding (Section 4.6, Section 5.4, Section 6.4, Section 7.6, Section 8.6, Section 9.7, Section 10.7, Section 11.6) and a principled framing in Section 1.4. This section consolidates them, organizes them by category, and states what each implies for downstream use of the findings.

Scope Limitations

Single dataset. All measurements are on CIFAR-10. We have no direct evidence that the findings transfer to other image datasets, non-image modalities, or higher-resolution data. The Phase 1 design (Section 12.3) extends the dataset axis but Phase 0 does not.

Bounded scale. Maximum model size is 56,391,434 parameters (CNN at w = 512, d = 4). Maximum width is 512; maximum depth is 32. LLM-scale results are not addressed by this paper. Whether any of our findings hold at $10^{}$ or 10^{11} parameters is an open question.

From-scratch training only. All models in this study are trained from random initialization. We have not measured pretrained models, fine-tuned models, or models initialized from any prior checkpoint. The Section 8 dynamics findings, in particular, are about training trajectory in a from-scratch setting.

Single optimizer and schedule. All experiments use a fixed optimizer (Adam with consistent hyperparameters) and a fixed training schedule (100 epochs). We have not varied learning rate, batch size, weight decay, or training duration as primary axes. The pre-specified Phase 1 protocol does not include these as primary axes either; they are reserved for a separate study.

Estimator Limitations

Finite-batch ceiling. The InfoNCE estimator at B = 512 is upper- bounded by log B = 6.238 nats. Mean MI(X; Z) in our data ranges 5.46 to 5.78 nats (90-93% of the ceiling), so the estimator is operating in a regime where it cannot resolve differences among architectures with true MI > 6.2 nats. This is the single largest interpretive limitation on the Section 4 width-scaling result.

Permuted-pair floor. The estimator returns non-zero values on shuffled (X, Z) pairs, with mean κ_{perm} scaling 0.083 to 0.011 across our widths. We separate this floor from the signal explicitly

(Section 2.3) and document its presence in every analysis where it matters. What we cannot do is reduce the floor; it is a property of the estimator-batch combination.

Architecture-dependent floor. Section 5.2 documented that the estimator floor (κ_{perm}) varies by architecture (5.20 to 5.76 nats at the canonical slice). This means cross-architecture comparisons of raw κ mix two effects: information content and floor. The Section 5 architectural constants are reported with this caveat; the Section 9 IB ratio (where the floor's d_z effect cancels) avoids the issue.

Single estimator family. We use one InfoNCE variant (separable cosine critic, B = 512). Results may differ with binning estimators, MINE/NWJ variants, or normalizing-flow-based estimators. We have not tested cross-estimator robustness in this paper.

- **Critic train/evaluation split.** The InfoNCE critic in this work is trained and evaluated on the same data batches; we did not perform held-out InfoNCE evaluation with separate train/eval splits for the critic. Phase 1 will include explicit critic train/eval separation to bound any overfitting contribution to κ measurements. ## Interpretive Limitations

Phenomenological vs. mechanistic. The Section 4 width law is a phenomenological fit (R^2 > 0.9999) whose mechanistic interpretation is contested by Section 4.2's own decomposition. We do not claim that the fit reveals a law of neural information; we claim it is a useful empirical regularity in our scope. The same caveat applies to Section 5 architecture constants and Section 6 depth exponents to a lesser degree.

Cannot identify causality. All correlations reported are descriptive. The Section 9 r(η_t, accuracy) = 0.685 says configurations with higher η_t achieve higher accuracy on average; it does not say that raising η_t *causes* accuracy to rise. Causality would require interventional experiments (e.g., pruning to fix η_t and measuring downstream accuracy) that are out of Phase 0 scope.

Cannot distinguish IB from related framings. The Section 9 finding that η_t correlates with accuracy is consistent with the IB principle but also with several alternative framings (mutual information criterion in feature selection; rate-distortion under specific distortion measures; etc.). We have not designed experiments to disambiguate these. We frame the result as "compatible with IB" rather than "IB proven."

Cannot generalize from observation. Statements that our findings "would also hold" in some other condition (different dataset, scale, training regime) are conjectures, not implications. Phase 1 (Section 12.3) is the appropriate vehicle for testing them.

Methodological Limitations

Pre-specification is incomplete. The σ = 0.1 perturbation, the SANITY_THRESH = 0.10 sanity rule, and the 5% compression-phase threshold were all pre-specified. The aggregation rules and statistical methods (Section 2.5, Section 2.6) were also pre-specified. *However*, the choice of which findings to feature in this paper, vs. which to relegate to appendix, was made after seeing the results. We have not run a double-blind analysis. The Section 12.4 falsifiable predictions are the post-hoc commitment that closes this gap going forward.

No external replication. All Phase 0 results were produced by a single research effort (one author, one compute environment). We have not had an independent group re-run the protocol on

the same dataset. Phase 1 explicitly invites such replication.

Limited dynamics scope. Section 8 reports training dynamics for 3 of 8 architectures (CNN, MLP, Pre-LN), only at d = 4, only at w $\in \{64, 256\}$. The compression-phase null result (0/63 runs) is consequently of narrow scope. We do not claim this null result extends to other configurations.

Layerwise d_z confound. Section 11.3 documented that CNN's per-layer d_z varies (256 $\rightarrow$ 1024 $\rightarrow$ 256), so its layerwise κ comparisons partially reflect architectural d_z structure rather than information flow per se. The other 7 architectures have constant d_z = 256 across taps; CNN is unique in this respect within our set.

What This Means for Use of Our Findings

We summarize, for prospective users, which findings transfer most safely out of our scope and which require strong caution.

Most safely usable: - The Section 7 phase-transition finding (residual-free architectures collapse at width-dependent d_c) is corroborated by accuracy and loss, neither of which depends on the InfoNCE estimator. Practitioners should treat the Section 7 d_c values as approximate but qualitatively reliable for the same dataset and training regime. - The Section 10 inverse-design forward model achieves <1% held-out median error for 6 of 8 architectures within our (w, d) box. Practitioners who need to size networks for a target κ on CIFAR-10, in our architecture set, can use the per-architecture fits. - The Section 9 IB-ratio finding (η_t correlates with accuracy in 7/8 architectures) is mathematically d_z-invariant and replicates at three aggregation levels (pooled, per-arch, group-mean). This is the most theoretically clean finding, though it is still on a single dataset.

Use with caution: - The Section 4 width law ($\kappa \propto$ w^(-1)) is highly accurate within our data but its meaning is contested by our own Section 4 decomposition. Use as a forward model is fine; interpret as "law of information capacity" is not. - The Section 5 architecture rankings are stable across two fits but may partially reflect estimator floor differences across architectures. Do not infer absolute architectural information capacity from C_arch. - The Section 6 depth exponent split (6 invariant + 2 sensitive) is consistent with Section 7 phase transitions but the magnitude of $\widehat{\beta}_d$ for individual architectures is sensitive to the (w, d) range over which it is fit.

Do not use: - These findings are for from-scratch CIFAR-10 training at our scale. Pretrained models, larger datasets, larger scales, and other modalities are explicit out-of-scope. - The Section 11 layerwise patterns are qualitative observations within a single canonical configuration. They should not be relied upon for architectural decisions outside that configuration. - Do not use the Phase 0 results as evidence for chip-level utilization, accelerator optimization, NPU/ASIC allocation, energy-efficiency claims, memory-bandwidth planning, or hardware co-design. The hardware-aware implications in Section 12 are explicitly future work, not present results.

We close Section 13 with the same point that opens Section 1: this paper is **Phase 0**. It is intended as a baseline, not a final answer. Users of our findings should weigh them as a baseline accordingly: useful for orientation, useful for falsification by Phase 1 results, useful for collaboration on the open questions of Section 12.4. Not useful as a substitute for the verification their own application would require.

Conclusion

What This Paper Is

This is an empirical paper. It does not propose a new theory of neural information processing, and it does not claim to have discovered universal laws. It specifies a measurement protocol with documented controls (Section 2), produces a reproducible body of measurements (Section 4-Section 11), and distinguishes findings that survive the protocol's controls from those that do not (Section 4.6, 6.4, 8.5). The patterns we observe across eight architectures suggest that the measurement methodology may generalize; that suggestion is the subject of the companion ICON framework specification, released alongside this paper.

Two of our findings — phase transitions in residual-free architectures (Section 7) and the d_z-invariant correlation between the IB ratio and accuracy (Section 9) — survive the Section 2 estimator caveats by independent arguments (multi-metric corroboration in Section 7, exact d_z cancellation in Section 9). Two others — within-config training dynamics (Section 8) and the inverse-design forward model (Section 10) — are robust within their stated scope and offer practical utility. Three further findings — the width-scaling law (Section 4), the architecture constants (Section 5), and the layerwise patterns (Section 11) — are reported as phenomenological observations whose mechanistic content is contested by the same data that establishes their phenomenology.

This separation is the paper's central organizing principle. It is the reason Section 4, despite being our headline $R^2 > 0.9999$ result, is *not* the section we ask readers to take away as proven, and Section 7 and Section 9, despite being more technically modest in scope, *are* the findings we offer with confidence.

What This Paper Establishes Confidently

We establish, within our scope (CIFAR-10, B = 512, 8 architectures, 2,241 records):

- Residual-free deep architectures undergo width-dependent catastrophic collapse at quantifiable critical depths d_c, with four independent metrics agreeing on d_c (Section 7).
- The information-bottleneck ratio $\eta_t = \kappa_{\text{task}} / \kappa_{\text{input}}$, which is algebraically free of the per-dimension normalization, correlates with test accuracy across architectures, configurations, and within individual configurations (Section 8, Section 9, Section 11).
- Within-configuration training increases κ_{task} while leaving κ_{input} approximately constant; we observe no strict information-bottleneck compression phase at our pre-specified 5% threshold (Section 8).
- A two-variable forward model $\kappa(w, d)$ extrapolates to held-out data with median 1.75% error in 5-fold cross-validation, enabling proof-of-concept for inverse design within the measured grid (Section 10).

We do *not* establish a universal information-density law, and we do not claim that our scaling exponents reflect architecture-intrinsic constants. The data permits a phenomenological reading; it does not permit a mechanistic one without measurements at larger batch sizes.

What Comes Next

Phase 1 of ICON (Section 12) will test four falsifiable predictions derived from the Phase 0 baseline. We will report Phase 1 results regardless of whether they confirm or contradict Phase 0 — the value of a baseline is in being falsifiable, not in being unchallenged.

The Phase 1 design (larger batch sizes, multiple datasets, four additional architectures, focused tests at 1B parameters) is beyond what our compute budget supports as a single research effort. We release the full Phase 0 protocol, raw data, and analysis scripts; we explicitly invite collaboration; we treat replication and extension as contributions to the open ICON measurement program.

Closing Statement

What we offer here is a careful Phase 0 baseline of an empirical program, not a closed body of results. The findings we report with confidence (phase transitions, IB ratio, training dynamics, inverse design) are robust within their scope and offer practical and theoretical entry points for follow-up work. The findings we report phenomenologically (width scaling, architecture constants, layerwise patterns) describe what we measured but require Phase 1 to interpret.

We expect the framework of Phase 0 — pre-specified protocols, explicit estimator-bias decomposition, evidential framing of findings, and a stated commitment to publish Phase 1 results regardless of outcome — may prove useful as a model for empirical work in this area, independent of the specific findings reported here.

The release is structured to remain open: open to being measured against, reproduced, extended, falsified, reinterpreted, refined, or simply read. We do not assume which form of engagement will prove most useful, and we do not commit to any single theoretical reading of what κ represents. Whoever finds the released measurements, protocols, or data useful — for whatever purpose, from whatever vantage — is welcome to use them.

Declaration of Generative AI Use

In the preparation of this paper, the author used Anthropic's Claude across separate sessions during 2025-2026 for assistance with: translation from Korean drafts, editorial refinement, bibliographic verification against primary sources, LaTeX format conversion, internal consistency checking, and writing of Python verification scripts that recomputed every numerical claim from the raw JSONL data. All conceptual content (experimental design, observations, analysis framework, framing decisions) originates with the author, who reviewed all AI-assisted output, verified all numerical claims against raw data, verified all citations against primary sources, and takes full responsibility for the content. In accordance with arXiv policy, AI tools are not listed as authors.

References

Companion work by the present author

- Park, J. (2026). *Information Flow and Self-Reference Across Substrates: On Self-Reference, Substrate, and the Need for Measurement*. Preprint released as a companion to the present empirical paper.

Code, data, and reproducibility materials.

- Phase 0 empirical reproduction package: https://github.com/joonhai-official/icon-empirical (release v0.1.0; Zenodo DOI: https://doi.org/10.5281/zenodo.20184000)
- Companion ICON framework specification and reference implementation: https://github.com/joonhai-official/icon (release v0.1.0; Zenodo DOI: https://doi.org/10.5281/zenodo.20184015)

The references below cite all in-text citations. Final formatting will follow the target venue's style at submission. Items in **bold** are those cited multiple times across the paper.

Information theory in neural networks

- **Tishby, N., & Zaslavsky, N.** (2015). Deep learning and the information bottleneck principle. *2015 IEEE Information Theory Workshop (ITW)*, 1-5. arXiv:1503.02406.
- Tishby, N., Pereira, F. C., & Bialek, W. (1999). The information bottleneck method. *Proc. of the 37th Annual Allerton Conference on Communication, Control and Computing*, 368-377.
- Shwartz-Ziv, R., & Tishby, N. (2017). Opening the Black Box of Deep Neural Networks via Information. arXiv:1703.00810.
- **Saxe, A. M., Bansal, Y., Dapello, J., Advani, M., Kolchinsky, A., Tracey, B. D., & Cox, D. D.** (2018). On the Information Bottleneck Theory of Deep Learning. *International Conference on Learning Representations (ICLR 2018)*.
- Goldfeld, Z., van den Berg, E., Greenewald, K., Melnyk, I., Nguyen, N., Kingsbury, B., & Polyanskiy, Y. (2019). Estimating Information Flow in Deep Neural Networks. *Proc. of the 36th International Conference on Machine Learning (ICML)*, PMLR 97:2299-2308. arXiv:1810.05728.

Mutual information estimation

- **van den Oord, A., Li, Y., & Vinyals, O.** (2018). Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748. [InfoNCE original paper]
- Belghazi, M. I., Baratin, A., Rajeswar, S., Ozair, S., Bengio, Y., Courville, A., & Hjelm, R. D. (2018). Mutual Information Neural Estimation. *Proc. of the 35th International Conference on Machine Learning (ICML)*. arXiv:1801.04062. [MINE]
- **Poole, B., Ozair, S., van den Oord, A., Alemi, A., & Tucker, G.** (2019). On Variational Bounds of Mutual Information. *Proc. of the 36th International Conference on Machine Learn-*

ing (ICML), PMLR 97:5171-5180. arXiv:1905.06922. [InfoNCE bound analysis, log B ceiling]

Scaling laws

- Hestness, J., Narang, S., Ardalani, N., Diamos, G., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., & Zhou, Y. (2017). Deep Learning Scaling is Predictable, Empirically. arXiv:1712.00409.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., & Sifre, L. (2022). Training Compute-Optimal Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS 2022)*. arXiv:2203.15556. [Chinchilla]

Residual / normalization architectural literature

- **He, K., Zhang, X., Ren, S., & Sun, J.** (2016). Deep Residual Learning for Image Recognition. *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778. arXiv:1512.03385.
- Hayou, S., Doucet, A., & Rousseau, J. (2019). On the Impact of the Activation Function on Deep Neural Networks Training. *Proc. of the 36th International Conference on Machine Learning (ICML)*, PMLR 97:2672-2680.
- **Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., & Liu, T.-Y.** (2020). On Layer Normalization in the Transformer Architecture. *Proc. of the 37th International Conference on Machine Learning (ICML)*, PMLR 119:10524-10533. arXiv:2002.04745. [Pre-LN vs Post-LN]
- Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., & Chao, L. S. (2019). Learning Deep Transformer Models for Machine Translation. *Proc. of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*, 1810-1822. arXiv:1906.01787.

InfoNCE estimator and finite-batch effects

- McAllester, D., & Stratos, K. (2020). Formal Limitations on the Measurement of Mutual Information. *Proc. of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 108:875-884. arXiv:1811.04251.
- Song, J., & Ermon, S. (2020). Understanding the Limitations of Variational Mutual Information Estimators. *International Conference on Learning Representations (ICLR 2020)*. arXiv:1910.06222.

Related Work and Background

ICON does not arise in isolation. Its methodology is related to prior work on the Information Bottleneck principle (Tishby & Zaslavsky, 2015) and its application to deep networks (Saxe et al., 2018; Goldfeld et al., 2019); to mutual-information estimation via contrastive lower bounds (van den Oord et al., 2018; Belghazi et al., 2018; Poole et al., 2019); to signal-propagation and edge-of-chaos analyses of neural networks (Schoenholz et al., 2017; Hayou et al., 2019); to residual network trainability (He et al., 2016; Xiong et al., 2020); to phase-transition-like phenomena in deep learning; to energy-based and Hopfield/Boltzmann lineages (Hinton & Salakhutdinov, 2006); to representation analysis and mechanistic interpretability; and to empirical scaling laws in deep learning (Kaplan et al., 2020; Hoffmann et al., 2022). Our contribution is to operationalize these concerns into a reproducible information-flow measurement protocol and a Phase 0 empirical baseline across eight architecture families.

Methodology

- Glorot, X., & Bengio, Y. (2010). Understanding the Difficulty of Training Deep Feedforward Neural Networks. *Proc. of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9:249-256.
- Wilcox, R. R. (2011). *Introduction to Robust Estimation and Hypothesis Testing* (3rd ed.). Academic Press. [Bootstrap CI]
- Stone, M. (1974). Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society, Series B*, 36(2), 111-147. [k-fold CV]

Appendix A: Architectures and Parameter Counts

The eight architectures studied. All implementations follow standard recipes; the exact code is released alongside the data (Section 2.7). Parameter counts at the canonical (w=256, d=4, ReLU) configuration:

Table A.1. Architecture parameter counts at the canonical configuration.

Architecture	n_params	Description
boltzmann	792,330	Restricted Boltzmann machine, single hidden layer
serial	1,065,482	Residual-free MLP-style stack
parallel	1,066,510	4 parallel branches, concatenated
gru	1,593,866	GRU over patch sequence
mlp	2,148,874	Standard MLP with skip connections
Trans (Pre-LN)	3,181,578	Pre-LN Transformer encoder
Trans (Post-LN)	3,181,578	Post-LN Transformer encoder

Architecture	n_params	Description
cnn	26,891,018	VGG-style convolution stack with channel doubling

CNN's larger parameter count reflects its convolutional kernel structure; the comparable d_z varies across layers (256 → 512 → 1024 → 256, see Section 11.3).

The κ measurement is taken at the pre-classifier representation `ffn_out`.

Appendix B: σ-Sweep (Robustness to Measurement Perturbation)

The κ measurement uses isotropic Gaussian noise of standard deviation σ = 0.1 added to the representation Z before InfoNCE estimation. We sweep σ across 12 values to verify the choice does not materially affect downstream results.

Filter: F_sigma_fine (results_p6/), n = 260 records. **Configuration**: w=256, d=4, ReLU per arch.

Table B.1. σ-sweep CV per architecture across $\sigma \in \{0.001, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0\}$.

Architecture	CV(κ_{input})	σ* (peak)	Pattern
mlp	0.25%	0.1	Flat across $\sigma \in [0.001, 1.0]$
Trans (Pre-LN)	0.64%	5.0	Slight rise at large σ
boltzmann	0.90%	0.02	Weak interior peak
Trans (Post-LN)	0.94%	10.0	Monotone-rising at boundary
parallel	0.96%	10.0	Monotone-rising at boundary
gru	1.07%	5.0	Interior peak
cnn	1.84%	0.5	Interior peak; sweep terminates at σ=0.5 due to numerical issues
serial	2.23%	10.0	Monotone-rising at boundary

For all 8 architectures, the κ value at our chosen σ = 0.1 is within 2.5% of the peak σ value. The σ choice does not meaningfully change any of our findings.

Appendix C: Temperature Sweep (Boltzmann and Pre-LN)

For the two architectures with explicit temperature parameters in their sampling/attention, we vary T $\in \{0.01, 0.1, 1.0, 10.0, 100.0\}$.

Filter: F_temp (main), n = 255 records.

Table C.1. Boltzmann κ_{input} by (width, T), CV across the T sweep.

width	T=0.01	T=0.1	T=1.0	T=10.0	T=100.0	CV
64	0.0899	0.0910	0.0917	0.0922	0.0896	1.11%
256	0.0230	0.0231	0.0233	0.0231	0.0221	1.86%
512	0.01156	0.01162	0.01168	0.01161	0.01106	1.97%

Table C.2. Pre-LN κ_{input} by (width, T).

width	T=0.01	T=0.1	T=1.0	T=10.0	T=100.0	CV
64	0.0879	0.0878	0.0893	0.0895	0.0892	0.82%
256	0.02261	0.02261	0.02265	0.02252	0.02227	0.61%

Both architectures show CV $\leq$ 2% across the full T range. The 100x range covered by the sweep produces very small κ_{input} changes, consistent with both architectures being approximately T-invariant in our scope.

We did not run T-sweeps on the other 6 architectures (their architectural designs do not involve a comparable temperature parameter).

Appendix D: Sanity-Failing Records (Descriptive)

The Section 2.4 sanity rule (κ_{perm} < 0.10) excludes records where the InfoNCE estimator's finite-batch floor dominates. Excluded records are descriptively summarized here.

Total sanity-failing records: 336 / 1,405 (23.9%) in the main results file.

Table D.1. Sanity-failing records by width.

width	sanity-failing records
16	168 (100%)
32	168 (100%)
64	0 (0%)
128	0 (0%)
256	0 (0%)
512	0 (0%)

Mean κ_{perm} in failing records: 0.236 (range [0.139, 0.328]).

The failure pattern is exclusive to small widths (16, 32). At small d_z, the InfoNCE estimator returns large κ_{perm} values (the floor in per-dimension form is roughly log B / d_z, which is 0.39 at d_z=16 and 0.19 at d_z=32). These records' κ values are therefore not distinguishable from the floor. We exclude them from primary analyses.

Appendix E: Seed Reproducibility

For each (arch, w, d, activation) configuration with at least 3 seeds in Φ_{default}, we compute the coefficient of variation (CV) of κ_{input} across seeds.

n_configs: 208.

Table E.1. Seed-replicate coefficient of variation (CV) statistics for κ_{input} across configurations.

Statistic	CV(κ_{input})
p25	0.088%
median	0.145%
p75	0.245%
p90	0.367%
max	3.702%

The median CV across configurations is 0.145%, indicating that the κ measurement is highly reproducible across random seeds.

The five highest-CV configurations are concentrated in transformer_postln at large widths (w=512), which are precisely the configurations near the Section 7 phase transition for that architecture. This is consistent with phase-transition regions exhibiting larger fluctuations.

Top 5 highest-CV configurations: - transformer_postln, w=128, d=16, ReLU: CV = 3.70% - transformer_postln, w=512, d=4, tanh: CV = 3.63% - transformer_postln, w=512, d=4, ReLU: CV = 3.06% - serial, w=256, d=8, ReLU: CV = 2.27% - transformer_postln, w=512, d=4, GeLU: CV = 1.73%

Appendix F: Saturation, Wall Time, and Reproducibility

Saturation distribution

A record is flagged `saturated` if its measured $\kappa_{\text{input}} \cdot d_z$ exceeds log B = 6.238 nats (the InfoNCE ceiling).

Total saturated records: 130 / 1,405 (9.3%) across all experiment types in the main results file.

Table F.1. Saturated records by architecture (saturation: $\kappa_{\text{input}} \cdot d_z \geq \log B$).

Architecture	saturated records
boltzmann	127
mlp	3
(others)	0

Table F.2. Saturated records by width.

width	saturated / total
16	0 / 168 (0.0%)
32	0 / 168 (0.0%)
64	0 / 267 (0.0%)
128	0 / 168 (0.0%)
256	61 / 412 (14.8%)
512	69 / 222 (31.1%)

width	saturated / total

Boltzmann at large widths dominates the saturation distribution, with 127/130 saturated records. This is consistent with RBM's per-dimension representations having lower variance than other architectures, allowing InfoNCE to push κ closer to the ceiling. We exclude saturated records from Φ_{strict} (Section 10) but include them in Φ_{default} for the width-law fits (Section 4) — the Section 4 fits use width-averaged means in which saturation enters as a minor positive bias.

Wall time

Total wall time across all 1,405 records (main file) and 836 records (P6 file): **27.79 hours** (1.16 days). Per-record mean is 44.7 seconds. The largest configurations (CNN at w=512, d=4) account for the upper quartile of per-record wall time.

Reproducibility checklist

The following are released alongside the paper:

1. **Raw data**: `results/full.jsonl` (1,405 records), `results_p6/full.jsonl` (836 records). sha256 hashes published in Section 2.7.
2. **Schema**: A documentation file describing all 28 fields per record.
3. **Analysis scripts**: The released Python analysis scripts regenerate every numerical claim. Each script implements the Section 2 filters, aggregations, and statistical methods.
4. **Training code**: Implementations of the 8 architectures and the InfoNCE estimator. Random seeds are recorded; data loaders use fixed shuffling.
5. **Pre-specification record**: A single document specifying that $\sigma = 0.1$, SANITY_THRESH = 0.10, and the 5% compression-phase threshold were chosen prior to running the experimental sweep.

A reproducer with access to the same hardware can re-derive every value in this paper from the raw JSONL files. A reproducer running the training code from scratch should expect to match each cell within the seed CV documented in Appendix E (typically < 0.2%).

The release is structured to support replication, extension, and falsification (Section 12.4 Predictions 1-4). We invite all three.